university of koblenz
Computer Science

Faculty 4: Computer Science

Institute for Software Technology

Institut für Softwaretechnik

Supporting the Specification of Fairness Requirements

Master's Thesis

in partial fulfillment of the requirements for
the degree of Master of Science (M.Sc.)
in Web and Data Science

submitted by

Rahul Thiruvengadam

Mat. Nr. 221100603

First supervisor: Dr. Qusai Ramadan
Institute for Software Engineering

Second supervisor: Katharina Größer
Institute for Software Engineering

Koblenz, September 2024

Eidesstattliche Erklärung:

Hiermit bestätige ich, dass die vorliegende Arbeit von mir selbständig verfasst wurde und ich keine anderen als die angegebenen Hilfsmittel – insbesondere keine im Quellenverzeichnis nicht benannten Internet–Quellen – benutzt habe und die Arbeit von mir vorher nicht in einem anderen Prüfungsverfahren eingereicht wurde.

	Ja	Nein
Mit der Einstellung der Arbeit in die Bibliothek bin ich einverstanden.	☑	☐

Koblenz, 27.09.2024

.....

(Ort, Datum) (Unterschrift)

Acknowledgment

I would like to sincerely thank my supervisors, Dr. Qusai Ramadan and Katharina Größer, for their guidance, constant support, and constructive feedback throughout the course of this thesis. Your insights and expertise were invaluable in shaping its direction and quality, and I am truly grateful for having had the opportunity to work under your mentorship.

On a personal note, I would like to express my gratitude to my family and friends, who have been a constant source of strength during challenging times. Their unwavering support and motivation helped me stay focused and determined.

Abstract

The growing significance of software systems in decision-making processes across various sectors such as healthcare, finance, hiring, and criminal justice has raised concerns regarding the fairness of these systems. Traditional approaches to software development often overlook fairness, especially in the early stages, such as Requirements Engineering. This thesis aims to address this gap by introducing three significant contributions: a comprehensive glossary of fairness concepts, a conceptual model for fairness, and a novel fairness template. The glossary provides clear definitions of fairness-related terms, ensuring consistent understanding among stakeholders. Building on this, the conceptual model offers a structured framework for managing fairness-related risks throughout the software development lifecycle. Lastly, the fairness template enables the systematic specification of fairness-related requirements. These contributions establish a strong basis for integrating fairness considerations into the early stages of software development processes.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

In today's rapidly evolving, automated, and data-driven world, the issue of software fairness has emerged as a significant societal and ethical concern. As technology influences every aspect of modern-day life, software strives to foster personalized experiences. It plays a critical role in decision-making in healthcare, criminal justice, and recruitment [39]. The need for equitable treatment for individuals and groups has been of paramount concern. However, as numerous cases have revealed, this software is not devoid of bias and discrimination. For example, Amazon's recruiting engine, designed to streamline the process of reviewing resumes of job applicants and selecting candidates for job interviews, discriminated against women[1]. Another example is Google's translate software, which, when translating gender-neutral sentences from English to Turkish and then back to English, changes the profession's gender, indicating gender stereotypes [12]. Figure 1 shows that automatically translating text from a language without gendered pronouns into English can be influenced by societal biases.

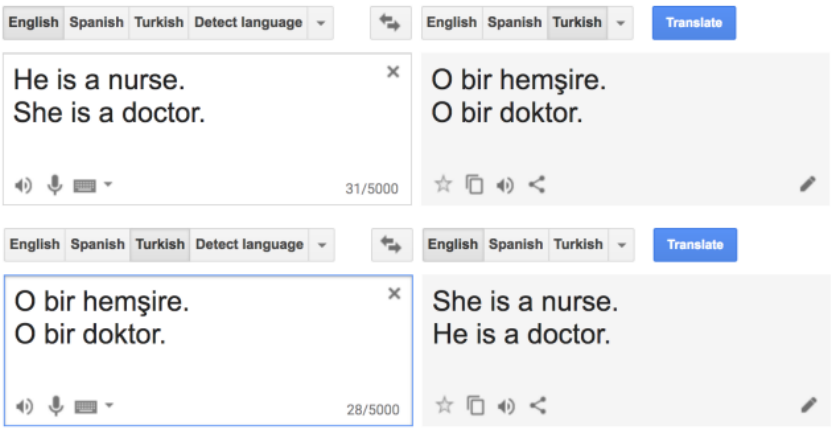

Figure 1: Example of gender bias in Google Translate [12]

While strides have been made to address fairness in the later stages of the software development lifecycle, a void persists in the requirements engineering phase [67]. This gap is best illustrated by the insidious *formalism trap*, exemplified by the COMPAS (Correctional Offender Management Profiling for Alternative Sanctions) software used within the American judicial system. COMPAS is a proprietary algorithm developed by Northpointe Inc. that is used to assess the risk of recidivism among criminal defendants. It analyzes various factors, including criminal history, age, employment status, and substance abuse history, to generate a risk score.

[1] https://www.reuters.com/article/us-amazon-com-jobs-automation-insight-idUSKCN1MK08G/

The score provided by COMPAS is intended to assist judges and parole boards in making informed decisions about pretrial release, sentencing, and parole.

ProPublica is an independent journalism organization that conducted a study in 2016 analyzing the effectiveness and fairness of COMPAS. They found that the algorithm had significant racial bias, with Black defendants being more likely to receive higher risk scores compared to white defendants, even when controlling for other factors. ProPublica's analysis raised concerns about the potential for algorithmic bias to exacerbate existing disparities in the criminal justice system[2].

However, developers of COMPAS claim that it significantly enhances efficiency by helping the parole board and judges make more informed decisions while promoting consistency by limiting the influence of subjective biases or personal judgments [79]. They also argued that it improved the overall accuracy of predicting recidivism rates [20]. On the other hand, critics of COMPAS raise concerns about the algorithm's lack of transparency and accountability, as well as its potential for perpetuating and exacerbating existing racial and socioeconomic disparities. They argue that the proprietary nature of COMPAS makes assessing its accuracy and fairness difficult, leading to opaque decision-making processes that may disproportionately harm marginalized communities [20].

COMPAS's flawed algorithm is a stark reminder of the perils of neglecting fairness considerations during requirements engineering, which can result in systemic biases and unjust outcomes. This issue could have been averted if precise requirements had been defined and validated by consulting legal experts or domain professionals before the development of COMPAS.

Motivation: The key motivation for this thesis is the evident lack of attention to fairness in the early phases of the software development lifecycle (SDLC), particularly in the Requirements Engineering phase. As shown in Figure 2, the SDLC is divided into early phases (Requirements Engineering, System Design, and Module Design) and later phases (Implementation, Unit Testing, and System Testing & Verification). Current fairness-related practices tend to focus on the later phases, such as implementation and testing, while neglecting the crucial initial stages where fundamental design decisions are made.

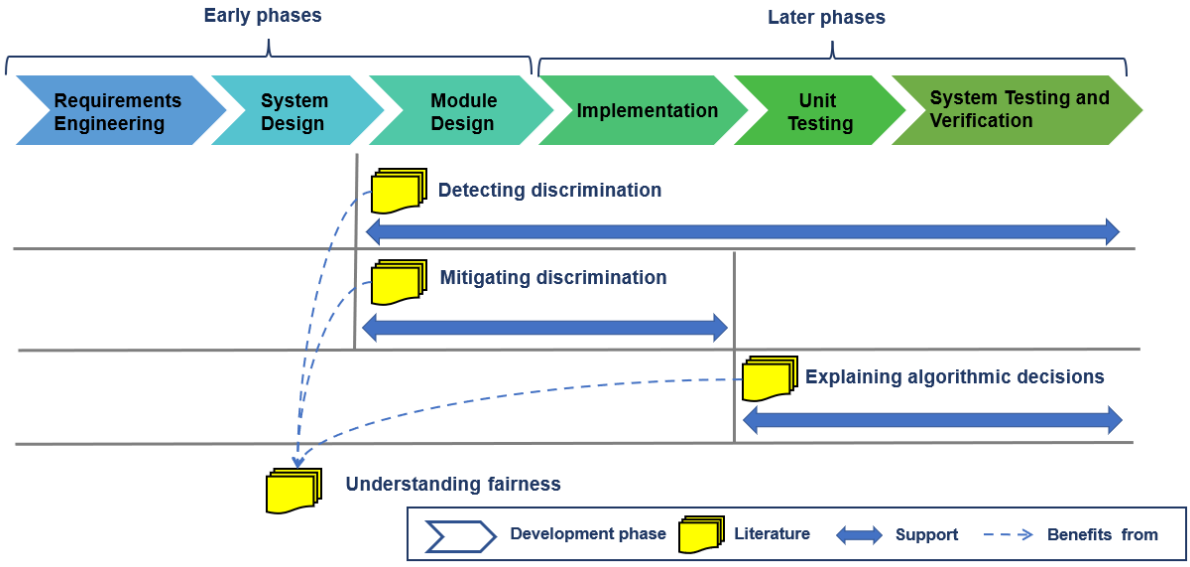

Figure 2: Fairness-Related Tasks Across the Software Development Lifecycle [66]

[2] https://www.propublica.org/article/how-we-analyzed-the-compas-recidivism-algorithm

In the early phases, like requirements engineering, there is a tendency to overlook fairness, focusing instead on functional and non-functional requirements [33]. This exclusion is significant because it sets the groundwork for potential biases that become deeply embedded in the system design. When fairness considerations are absent in these foundational stages, efforts to detect, mitigate, or explain discrimination in the later phases become reactive rather than proactive. Since software systems play a crucial role in influencing decisions that impact lives, it becomes imperative to address fairness not just at later stages but right from the outset during the requirements engineering phase. This thesis aims to address this gap by specifying fairness-related requirements during the requirements engineering phase, ensuring that fairness is integral to the software system from its inception. By doing so, we aim to build a robust foundation that facilitates fair and equitable software throughout its lifecycle.

1.1 Problem Statement

Specifying fairness-related requirements becomes challenging in the absence of a conceptual model. Without a conceptual model, there is a lack of a structured framework outlining fundamental concepts and their relationships. Experts may have to rely on their implicit knowledge to design a fair system; however, differences in shared understanding among them might inadvertently lead to biases[3]. For instance, consider the requirement in software development, *the hiring software should avoid discriminatory candidate selection.* This requirement does not provide clear criteria as to what actions or parameters define discrimination in candidate selection. Due to the absence of clear definitions or measurable indicators of discrimination, there is no objective way to assess or measure if the software adheres to this requirement. Different interpretations of discrimination could lead to subjective evaluations and challenges in ensuring compliance.

Another significant challenge in specifying fairness-related requirements is the lack of templates that explicitly incorporate fairness considerations. In software development, a template serves as a structured guide to define and document requirements for software systems, ensuring consistency and coverage of all essential aspects of software development [67]. However, these templates are generic and lack the necessary components to handle fairness-related requirements. For instance, the MASTER (Mustergültige Anforderungen - die SOPHIST TEmplates für Requirements) template, a well-known template for specifying requirements, typically emphasizes the functional and technical aspects but does not adhere to the fairness requirements of the system [31]. Without these fairness considerations, developing fair and equitable systems becomes challenging.

Challenge: To address the challenge of specifying fairness-related requirements, designing a conceptual model will help provide a structured framework to outline and understand the fundamental ideas, relationships, and tenets of fairness. A carefully designed conceptual model can address ambiguity and inconsistency in fairness-related terms and requirements, ensuring all stakeholders involved operate from a common perspective [70]. This conceptual model would establish a basis for standardizing fairness in software development, reducing the potential of prejudice or misinterpretation arising from unclear definitions.

However, designing this conceptual model can be challenging due to its complexity and multidisciplinary nature of integrating ethical, technical, legal, and social elements within software development [42]. The most prominent challenge is the lack of an established glossary that succinctly describes the concepts and their relationships pertaining to fairness [12]. While a

[3]Fairness in Machine Learning and ML Enabled Products

glossary defines terms like *bias*, *discrimination*, and *fairness*, it is insufficient when used alone because these terms may have multiple interpretations and could potentially lead to experts unintentionally working from different perspectives. For example, *Statistical Parity* and *Demographic Parity* can be used interchangeably to describe equal representation among demographic groups. Therefore, a conceptual model is necessary to bridge these gaps, ensuring a consistent and thorough approach to fairness in software systems.

In addition to this challenge, another significant hurdle is the need for a template that explicitly incorporates fairness considerations into the requirements engineering process. To address this issue, the MASTER template has been chosen to be customized to incorporate fairness-related terms. The MASTER template was selected for its established structure, adaptability, and ease of use [29]. Its structured approach makes it highly readable and user-friendly for new users to comprehend and define requirements. Its simple approach enables integrating fairness-related terms with the standard technical and functional requirements, ensuring consistency across the template [29]. Furthermore, its extensive use in industries that require detailed and rigorous requirements documentation makes it the perfect choice [71].

Thus, the dual challenge lies in both designing a robust conceptual model that clearly defines fairness-related concepts and relationships and customizing the MASTER template to ensure that fairness is a fundamental component of software requirements. A detailed glossary will be developed that includes fairness and related concepts, ensuring that all stakeholders have a common understanding. Using this glossary as a foundation, a conceptual model will be developed that provides a structured framework of how fairness considerations interact with the software development process, and it will also emphasize the relationship between these fairness concepts. This conceptual model will be used to customize the MASTER template to incorporate fairness as a critical component. This process helps bridge the gap between abstract ethical principles and practical software development, ensuring fair and equitable systems are created.

1.2 Research Question

The following research questions have been formulated to direct this thesis and address the fundamental concerns surrounding the specification of fairness-related requirements in software development, building on the challenges and gaps noted in the preceding sections:

Research Question 1 (RQ1): What are the fundamental concepts essential for the specification of fairness-related requirements in software development?

This research question identifies the core concepts pertaining to fairness; It aims to delve into the foundational ideas and principles related to specifying fairness-related requirements. A key outcome of this work will be the development of a comprehensive glossary that contains details on fairness and its related concepts, providing a common understanding for all stakeholders in the software development process.

Research Question 2 (RQ2): How can the relationships between the identified concepts be effectively expressed and structured in a conceptual model for specifying fairness-related requirements in software?

This research question seeks to understand how the identified concepts from RQ1 can be interconnected and expressed in a conceptual model. It addresses the challenge of creating a comprehensive framework that can guide the specification of fairness-related requirements in software.

Research Question 3 (RQ3): How can the MASTER template be modified effectively to incorporate explicit fairness conditions in software requirements?

This research question aims to investigate and identify how the existing MASTER template can be effectively modified to explicitly incorporate the fairness concepts identified in RQ1 and structured in RQ2. Additionally, it includes the extension of the MASTER meta-model to integrate these fairness considerations. The conceptual model developed in RQ2 will serve as a foundation to guide the necessary modifications, ensuring that fairness considerations are systematically integrated into the specification of software requirements.

Research Question 4 (RQ4): How can the modified MASTER template be practically applied and validated in a real-world scenario to assess its feasibility in specifying fairness-related requirements?

This research question focuses on defining fairness-related requirements using the modified MASTER template in a real-world test case. The goal is to evaluate how well the modified template supports the specification of fairness-related requirements in a practical context.

1.3 Thesis Contribution

This thesis focuses on the crucial gap in the software development process, specifically in the *requirements engineering* stage, where fairness considerations are often overlooked. The main contribution of this research is developing a systematic approach to incorporate fairness into the initial phases of software development.

Firstly, the thesis creates an extensive glossary of fairness-related concepts. This glossary serves as a fundamental guide, offering clarity and a common understanding amongst stakeholders. Having a shared understanding is crucial for stakeholders and developers to consistently ensure fairness in software systems.

Expanding on this glossary, the research presents a conceptual model designed to address fairness concerns in software development. Drawing inspiration from an established security risk management model, the conceptual model presented in this thesis illustrates the relationships among various fairness and security-related concepts, guiding the stakeholders and developers to systematically incorporate fairness into the software development process.

The thesis then focuses on adapting the current MASTER template to develop a customized fairness template. This modified template introduces additional elements pertaining to fairness, ensuring a clear definition of fairness requirements in the requirements engineering phase. Alongside the template, a corresponding meta-model is developed to provide a technical blueprint for implementing fairness requirements, adding a structured layer to the practical application of the template.

Finally, the thesis evaluates the feasibility of the modified fairness template through test cases. This evaluation provides insights into the practical utility of the proposed framework and highlights its potential to bridge the gap between theoretical fairness concepts and their implementation in software systems. Together, these contributions offer a comprehensive approach to embedding fairness into software development from its inception.

1.4 Thesis Structure

This thesis consists of ten chapters, each of which aids in understanding fairness and its integration into software systems. *Chapter 1: Introduction*, presents the motivation behind this research, outlining the critical gaps in fairness considerations during the SDLC, particularly in the requirements engineering phase. It also introduces the research questions that guide the thesis. *Chapter 2: Background Research*, presents the foundational knowledge necessary to approach this thesis. *Chapter 3: Methodology*, outlines the research methods used in this thesis. *Chapter 4: Systematic Literature Review*, presents the results of the literature review process. *Chapter 5: Information Extraction and Glossary Creation*, outlines the process of extracting fairness-related concepts from the reviewed literature and compiling a comprehensive glossary that informs the development of the conceptual model and fairness template. *Chapter 6: Conceptual Model*, introduces a structured framework developed to manage fairness risks in software systems. *Chapter 7: Fairness Template*, showcases a template that assists in defining fairness-related requirements. *Chapter 8: Test Case and Evaluation*, applies the fairness template to real-world test cases to evaluate its feasibility in generating fairness-related requirements. *Chapter 9: Related Work*, analyzes the current state of fairness research, highlighting the similarities and differences with the proposed work. *Chapter 10: Conclusion and Future Work*, summarizes the main contributions of the thesis, acknowledges its limitations, and proposes potential areas for future study.

Chapter 2

Background Research

This section provides a thorough overview of fairness in the context of software systems and their development. It begins by outlining the various *types of fairness*, emphasizing the different ways in which fairness is perceived in different contexts. Following this, it addresses the challenges in ensuring equitable outcomes in both traditional and machine learning (ML) systems, highlighting the intricacies of developing equitable systems. It also examines *fairness metrics* and *fairness techniques*, outlining ways to assess, evaluate, and promote fairness in a system's development process. Additionally, this chapter includes an *overview of the security risk management model*, which inspired the conceptual framework for fairness, developed in this thesis. Lastly, the *overview of the requirements template* and the *overview of the MASTER Template* outline existing approaches to structuring software requirements, providing the foundational context for the fairness template developed later in the thesis.

2.1 Overview of Fairness

In software systems, the term *Fairness* has multiple interpretations, each of which offers a unique perspective on what it means to achieve equality in automated decision-making processes. However, the most agreed-upon definition of fairness in software is that algorithms and automated decision-making processes function without any prejudice, ensuring that no individual or group is unfairly favored or disadvantaged based on protected attributes such as race, gender, age, or socioeconomic status [78]. These ideas of fairness are critical in fields where software has a significant impact on people's lives, such as healthcare, law enforcement, finance, and hiring [54]. Fairness in these systems must be upheld to prevent bias and discrimination from perpetuating, as doing so will have detrimental effects on society, ethics, and the law.

2.2 Types of Fairness

There are several types of fairness, each with distinct implications for how it is measured and achieved. Mentioned below are some popular types of fairness:

Individual Fairness - Ensures that similar individuals receive similar treatment or outcomes [12]. For example, suppose two loan applicants have very similar financial histories, credit

scores, and income levels. Individual fairness would require the bank's loan approval system to make similar decisions (approve or deny) for both applicants.

Group Fairness - Ensures that different groups (often defined by sensitive attributes such as race, gender, or age) are treated equitably [12]. For example, in a loan approval system, ensuring group fairness would involve ensuring that the loan approval rate for minority applicants is similar to the rate for non-minority applicants so that no particular group is unfairly put at a disadvantage.

Subgroup Fairness - Extends group fairness by ensuring equal treatment among all distinguishable subcategories within a protected attribute [45]. For example, among female loan applicants, subgroup fairness mandates that the system treat subgroups, such as women from various age groups or educational backgrounds in an equitable manner. Women under 30 with college degrees should have an approval rate that is similar to that of women over 30 with the same degree.

Counterfactual Fairness - A decision-making process would give the same outcome for an individual even if their sensitive attributes (such as race or gender) were different [50]. This definition aims to eliminate biases that arise from variables correlated with these attributes, ensuring decisions are not indirectly influenced by them. For instance, suppose an algorithm evaluates a loan application and approves it for a white applicant. To satisfy counterfactual fairness, the algorithm should also approve the loan if all the applicant's financial details remain the same, but their race is changed to another group.

Distributive Fairness - Ensures a fair distribution of outcomes across individuals or groups [24]. For example, in banking, distributive fairness might involve ensuring that loan approvals are distributed fairly among applicants of different socioeconomic statuses.

Procedural Fairness - Focuses on the fairness of the processes and methods that lead to outcomes, ensuring uniform and transparent implementation [24]. For example, a bank should have clear guidelines, explicit communication, and a consistent process for evaluating loan applications to ensure that all applicants are treated fairly.

Predictive Fairness - Ensures that predictions or decisions made by a system are equally accurate for all groups [53]. For example, a bank's credit scoring system is considered to have achieved predictive fairness when it predicts loan default risk equally well for both minority and non-minority applicants. For instance, if the system forecasts a 70% likelihood of default for two candidates from separate categories, the actual results should consistently match this probability for both.

Interactional Fairness - Ensures that the interactions between individuals and the software are fair and unbiased. This can include user interface design, feedback mechanisms, and customer support. For example, A bank's chatbot for customer service should offer equal levels of assistance and politeness to all users, regardless of their background or the language they speak.

Causal Fairness - Ensures that decisions are not influenced by prohibited causal relationships from protected attributes [47]. For example, a loan approval system demonstrates causal fairness when the decision to approve or deny a loan is based on legal considerations like income or credit history rather than being swayed by protected attributes like gender.

2.3 Key Factors Impacting Fairness in Software Systems

There are several challenges to ensuring fairness in software systems, both in traditional algorithms and Machine Learning. Although fairness concerns have long been present in software

development, the issue has grown more pressing with the reliance on machine learning, which is used to make critical decisions. Machine learning models are more prone to bias, especially societal biases or exclusion of a specific group, due to their heavily dependent nature on extensive datasets [42].

This thesis primarily focuses on specifying fairness-related requirements, but it is also essential to understand the broader factors that impact fairness in software systems. This section explores the critical elements that impact fairness in both traditional algorithms and machine learning models, particularly highlighting how machine learning's complexity and ubiquitous presence in critical applications exacerbate the issue. While these factors present systemic challenges, not all are directly tackled in this research.

Bias in Training Data - Bias in data is a significant challenge because the datasets used to train machine learning algorithms exhibit historical and societal biases [42]. Historical biases perpetuate existing prejudices. For instance, *sampling biases* arise when the data does not represent all relevant groups equally, leading to skewed outcomes [42]. *Label bias* occurs when the labels used in supervised learning are based on subjective human judgment [42].

Algorithmic Bias - Algorithms can introduce their own biases based on the way they handle and interpret data. This may be the result of model selection, features, or the optimization criteria [57]. For instance, algorithms that focus solely on maximizing accuracy may produce biased results if there are disparities in accuracy among various groups.

Defining Fairness - Defining fairness is complex because its meaning can vary significantly across different contexts, such as *equal opportunity*, *demographic parity*, or *individual fairness* [13]. Selecting the appropriate definition is context-dependent and may lead to disagreements. Each definition has its own trade-offs and implications, making it challenging to meet all fairness criteria simultaneously [13].

Transparency and Explainability - The intricate nature of many machine learning models, particularly deep learning models, often results in a lack of transparency, complicating the identification and correction of unfair practices [35]. Understanding why algorithms make decisions is essential as it allows for the detection of biases and ensures accountability [3]. However, balancing high performance with explainability can be quite challenging.

Regulatory and Legal Challenges - The ever-evolving legal and regulatory environment pertaining to fairness poses a challenge for developers. Ensuring compliance to a wide range of requirements, sometimes conflicting regulatory requirements across different jurisdictions adds complexity[1].

Technical Limitations - Dealing with bias in a complex model can be challenging without leveraging advanced tools and methodologies. Additionally, navigating the trade-offs between fairness and other model objectives is crucial to developing effective bias mitigation strategies without substantially compromising system performance [42].

Evaluation and Monitoring - Establishing suitable metrics and benchmarks for assessing fairness is difficult, as it necessitates careful consideration of the context and the specific fairness criteria being used [34]. To maintain continuous fairness, constant monitoring and modifications are necessary, especially as new data and scenarios emerge. This process demands dedicated resources and ongoing effort.

This thesis addresses factors related to *defining fairness* within software requirements and the *evaluation and monitoring* of fairness throughout the SDLC by developing a fairness-oriented

[1]What to do when your product must comply with conflicting requirements

conceptual model and fairness template. Additional considerations like *bias in training data*, *algorithmic bias*, *transparency and explainability*, and *regulatory concerns*, are recognized as important but are beyond the scope of this thesis. These broader issues are recognized as areas for future research and development in the field of fairness.

2.4 Fairness Measures

Fairness measures are either quantitative or qualitative metrics that are employed to evaluate how equitably a system functions among various groups or individuals [74]. Fairness measures offer a method to consistently identify and address inequalities, guaranteeing that software systems adhere to ethical norms. Various fairness measures concentrate on different aspects of fairness, ranging from evaluating groups to addressing individual treatment. Below are some common fairness measures, their definitions, and examples.

Statistical Parity / Demographic Parity - A decision-making process is fair if individuals from different demographic groups have similar probabilities of receiving favorable outcomes [57]. For example, a loan approval algorithm should grant loans at similar rates across different racial or gender groups, assuming equal qualification levels.

Equalized Odds - This measure demands that the true positive rate (TPR) and false positive rate (FPR) remain equal among various groups. It ensures that the model functions uniformly for every group [36]. For example, If the probability of accurately forecasting loan repayment (TPR) and mistakenly refusing a loan to an applicant who would have repaid it (FPR) is the same for all demographic categories, then a credit scoring model shows equalized odds.

Equal Opportunity - Special case of equalized odds where individuals who qualify for a specific outcome should have an equal chance of being selected, regardless of their demographic group [36]. For example, applicants with the same credit score, income, and financial history should have an equal chance of loan approval, irrespective of their race, gender, or other sensitive attributes. This ensures that qualified individuals are not disadvantaged by biases inherent in the system.

Conditional Demographic Parity - An extension of statistical parity. It ensures that parity is maintained not across the entire population, but within subpopulations that are defined by legitimate characteristics [16]. For example, for individuals making more than a specific income level, the acceptance rates should be equal for both minority and non-minority individuals.

Predictive Parity - Ensures consistency of positive predictive value (PPV) among various groups [77]. The PPV expresses the likelihood that someone who receives a positive prediction, such as loan approval, belongs to the positive class, that is, will repay the loan [77]. For example, A bank's loan approval model demonstrates predictive parity when the likelihood of loan repayment is the same for both male and female applicants after loan approval.

Calibration - A model is deemed calibrated if the predicted probability matches the actual outcomes for all groups [62]. For example, if a credit scoring model predicts a 70% likelihood of loan repayment for minority and non-minority applicants alike, then around 70% of both groups should indeed repay their loans. This guarantees calibration among different groups.

Treatment Equality - Evaluates the balance of false negatives (FN) and false positives (FP) ratios between various groups [8]. For example, treatment equality in a loan approval system would mean that the ratio of minority and non-minority applicants who are incorrectly denied loans (FN) compared to those incorrectly approved (FP) is equal.

2.5 Fairness Techniques

Fairness techniques are strategies employed by the system to ensure it operates free of any prejudice or bias. These methods can be used at various points in the model development process, prior to, during, or following the model's training, and hence, are primarily classified into pre-processing, in-processing, and post-processing [64]. Below are widely used fairness techniques, their definitions, and examples

Pre-Processing Techniques These techniques include altering the input data prior to model training to guarantee fair and unbiased learning [57]. By addressing biases in the data, these techniques seek to mitigate their impact throughout the model's training process. Mentioned below are a few pre-processing techniques:

- **Re-sampling** - Re-sampling equalizes the presence of various groups in the training data through oversampling minority groups or undersampling majority groups [43]. Doing so ensures that the model does not show preference towards the more dominant groups just because they appear more frequently in the dataset. For example, if the loan approval model's training data has a higher number of non-minority applicants than minority applicants, re-sampling methods can even out the dataset by increasing the number of minority applicants, helping to minimize biased predictions.
- **Re-weighting** - Re-weighting involves giving each dataset instance a different weight according to which group they belong to [43]. During training, this method guarantees that the learning algorithm assigns the same weight to the majority and minority groups [43]. For example, to reduce any historical bias in the data, the bank can assign higher weights to minority applicants.
- **Data Transformation** - To provide a fair representation, data transformation entails changing the characteristics in the training data [7]. Protected attributes or their proxies may be eliminated or modified to avoid prejudice while training a model [7]. For example, if a bank discovers that a zip code may be used as a proxy for race, it can modify this attribute to eliminate any discriminatory effects while maintaining its predictive value.

In-Processing Techniques These techniques are applied during the model training phase. These strategies modify the learning algorithm to include fairness constraints, enabling the model to make more equitable decisions according to the specified fairness criteria [57]. Mentioned below are a few in-processing techniques:

- **Fairness Constraints** – This entails integrating fairness criteria, such as equal opportunity and demographic parity, into the model's objective function while it undergoes training, to ensure that the model is optimized for both accuracy and fairness [21]. For example, a bank may set a constraint during loan approval model training that necessitates an identical true positive rate for minority and non-minority applicants. In this way, the model learns how to assess loans while adhering to fairness criteria.
- **Adversarial Debiasing** - This technique employs an adversarial approach to reduce the model's ability to predict sensitive attributes, such as gender or race, while still fulfilling the primary prediction purpose [83]. It guarantees that the model's choices are not as impacted by sensitive attributes [83]. For example, an adversarial network can be incorporated into the training process for a bank's credit scoring model to identify and reduce the model's dependence on sensitive attributes such as gender, leading to an equitable final model.

- **Regularization Techniques** - Regularization methods incorporate fairness-promoting components into the loss function of the model [21]. By penalizing the model for showing biased behavior, these terms help the model produce predictions that are more equitable. For example, during the development of a loan approval model, the bank could incorporate a regularization term to discourage significant disparities in approval rates among different demographic groups.

Post-Processing Techniques These techniques are applied once the model has completed its training. These strategies include adjusting the model's results in order to comply with fairness standards without changing the training data or the model itself [21]. Mentioned below are a few post-processing techniques:

- **Threshold Adjustment** - Changing decision thresholds for various groups can aid in equalizing results among those groups [36]. By adjusting various decision thresholds, the model can reach a specific fairness metric, such as equal opportunity or statistical parity. For example, to achieve statistical parity, a bank can modify the loan approval thresholds such that the approval rate for minority applicants is equal to that of non-minority applicants.
- **Output Perturbation** - In order to achieve fairness and mask biases, output perturbation entails introducing regulated noise to the outputs of the model [21]. This technique alters the final decisions in a way that satisfies fairness standards [21]. For example, after a loan approval model has made its decisions, a bank may make random adjustments to ensure that the percentage of approved loans for minority applicants better matches fairness objectives.
- **Reject Option Classification** - This technique allows the model to delay making a decision in situations of uncertainty, especially when fairness is a priority [7]. These choices can undergo a manual review to guarantee equitable results. For example, if a banking system's loan approval model lacks confidence in its decision for minority group applicants, it may pass these cases to human reviewers, who can use fairness guidelines to make a well-informed decision.

2.6 Overview of The Security Risk Management Model

The conceptual model shown in Figure 3, taken from the paper titled "*An Integrated Conceptual Model for Information System Security Risk Management Supported by Enterprise Architecture Management*", provides a comprehensive framework for managing security risks within organizations [56]. This model integrates Enterprise Architecture Management (EAM) with Information Systems Security Risk Management (ISSRM), providing a systematic approach to identifying, mitigating, and monitoring security risks across various system elements and business assets [56].

The core of the model involves integrating risk and security management, connecting risks to system elements and business assets. Important elements of the model consist of *risk* (potential events that may harm the system), *threats* (external or internal factors that exploit system weaknesses), *vulnerabilities* (the weaknesses that can be exploited), and *controls* (strategies used to reduce or eliminate risks). Together, these elements create a complete flow where risks are constantly evaluated and managed.

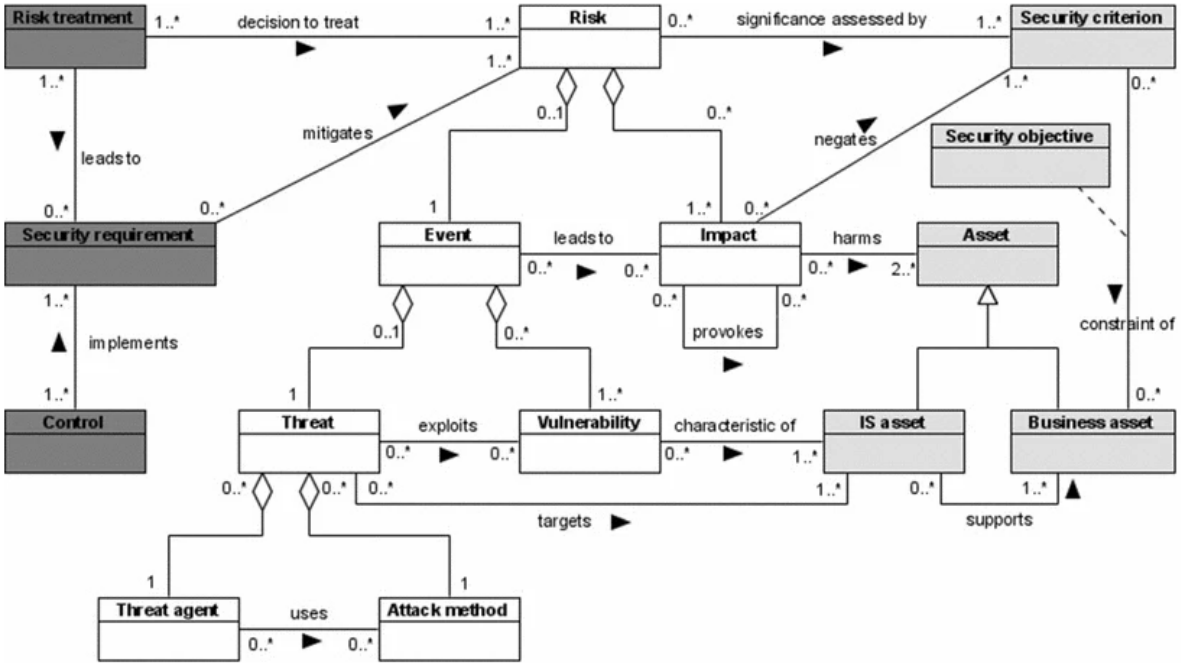

Figure 3: Security Risk Management Model [56]

The model's unique aspect is its connection to Enterprise Architecture Management (EAM). By connecting security management with the wider organizational structure, the model guarantees that all facets of the organization are taken into account when addressing security concerns. This connection enables security to be integrated at every level by mapping security control to specific elements of the organization's structure.

The process flow in the model operates cyclically, emphasizing continuous risk identification, monitoring, and mitigation. This architecture-driven approach gives it the flexibility to be incorporated in various industries and organizational settings, to meet the changing needs of the organization.

This conceptual model served as the inspiration for the development of the fairness conceptual model in this thesis, which is further detailed in Chapter 6. Similar to how the ISSRM-EAM model systematically manages security risks across an organization's architecture, the fairness model aims to manage fairness risks throughout the software development lifecycle. By adopting a structured approach to fairness, the fairness conceptual model ensures that fairness is addressed from the early stages of software development.

2.7 Overview of The Requirements Template

A requirements template is a blueprint that details the syntax of an individual requirement [71]. They are also referred to as generic, syntactical requirements templates because these templates outline the syntax of the requirement and do not detail the semantics [71]. Establishing guidelines for the creation of requirements ensures that every requirement follows a consistent structure, and this helps to prevent errors from the onset [71]. During the development process, this way of structuring requirements helps to decrease ambiguity and promote clarity.

Effective requirements engineering is crucial for the success of a software project. Identifying and resolving potential problems at an early stage of development can lead to considerable cost savings in comparison to making modifications later [63]. Defining precise and thorough requirements lowers the risk of misunderstandings between developers and stakeholders, reducing the need for expensive modifications caused by unfulfilled expectations [63]. Templates for requirements can outline both functional and non-functional requirements, including important elements like purpose, scope, constraints, and acceptance criteria. All the stakeholders involved benefit from this structured approach which guarantees uniformity and accountability throughout the software development lifecycle.

In the context of fairness, the structured and systematic nature of a requirements template serves as an inspiration for the fairness template. The fairness template expands on the idea of typical requirements templates by ensuring that a system performs as expected and that all users are treated equally. It presents components that directly pertain to fairness, such as fairness measures, fairness techniques, protected attributes, anti-protected attributes, and proxies, which are discussed further in Chapter 7. This modification of the usual requirements template allows for the uniform definition of fairness criteria during the entire software development lifecycle.

2.8 Overview of The MASTER Template

This thesis examines the set of templates called MASTER (Mustergültige Anforderungen - die SOPHIST TEmplates für Requirements), designed and developed by SOPHIST [29]. The reason for selecting the MASTER templates for examination was its thorough documentation and widespread application in real-world scenarios [30]. These templates address a broad range of requirement types and offer thorough instructions on organizing requirements in a systematic manner.

The MASTER template collection provides a predetermined structure for defining functional requirements, non-functional requirements, and conditions [29]. MASTER templates help guarantee that software requirements are precise, uniform, and consistent, thereby lessening the possibility of misunderstandings between stakeholders and developers [71].

Functional Requirements
The functional MASTER template is designed to facilitate the structured and clear specification of functional requirements within a software system [29]. It outlines the specific behaviors, actions, or functions a system must be able to perform. They specify what the system must do to satisfy the needs of the user, typically outlining user-system interactions and system reactions to various inputs [29].

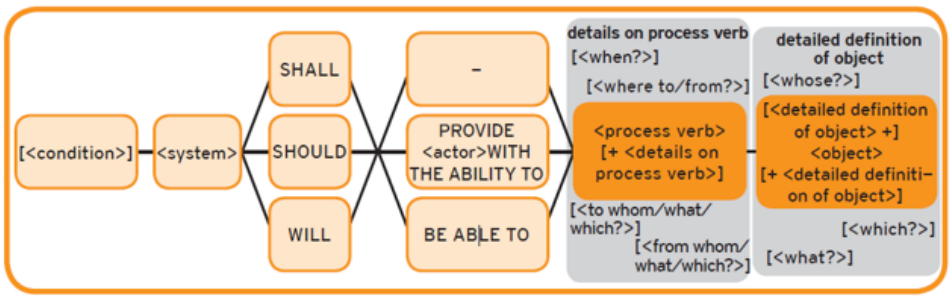

Figure 4: Functional MASTER Template [29]

Figure 4 presents the Functional MASTER Template, and the following example demonstrates how a requirement can be specified using this template in a banking scenario.
Example: The system SHALL PROVIDE the user WITH THE ABILITY TO schedule payments for registered payees up to 30 days in advance

Non-Functional Requirements
The non-functional MASTER templates are designed specifically to capture the non-functional aspects of a software system. Functional requirements specify the actions the system must take, whereas non-functional requirements outline how these actions are executed [29]. Non-functional requirements encompass aspects like performance, security, usability, and reliability, to guarantee the system functions efficiently and satisfies stakeholder needs. The MASTER collection provides three primary templates for specifying non-functional requirements: *PropertyMASTER*, *EnvironmentMASTER*, and *ProcessMASTER*.

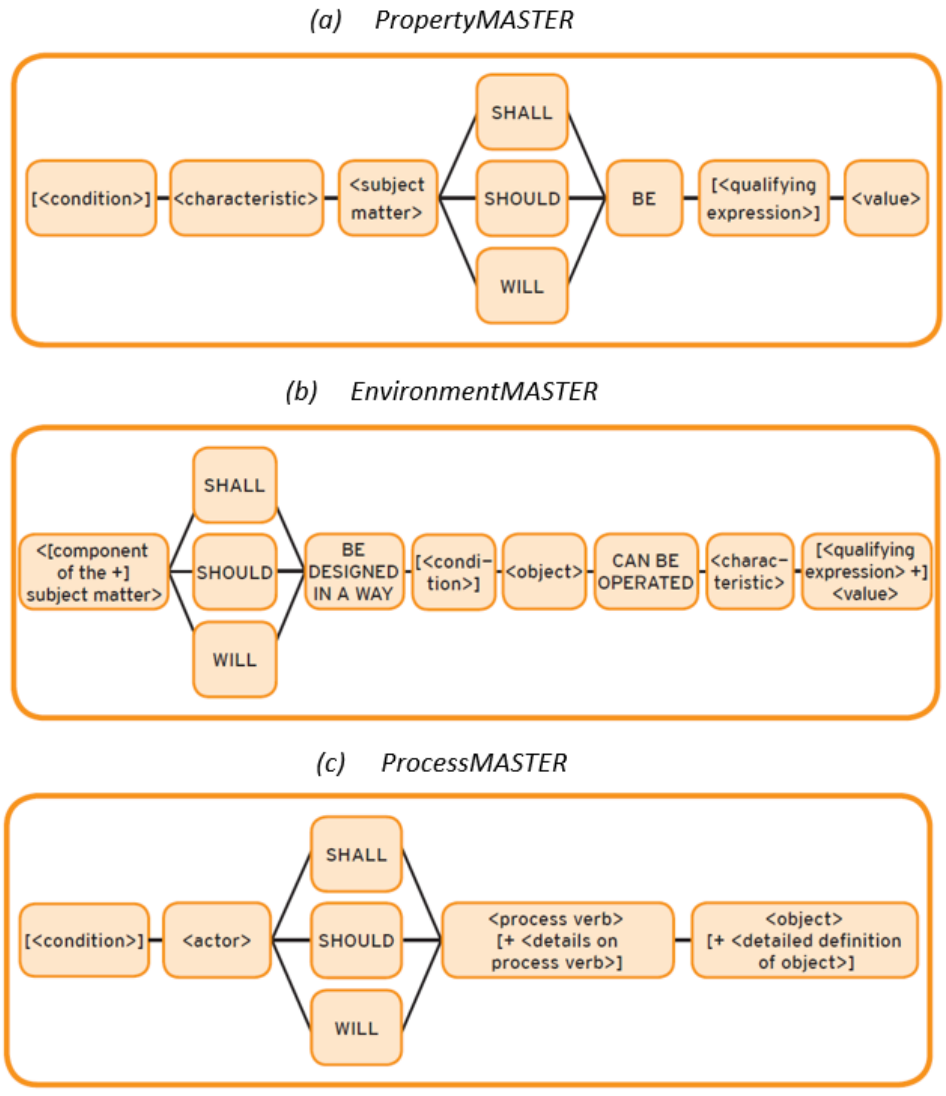

Figure 5: Non-Functional MASTER Template [29]

Figure 5 presents the Non-Functional MASTER Template. Below is a detailed explanation of these non-functional MASTER templates, followed by examples demonstrating how requirements can be specified using these templates in a banking scenario.

a) **PropertyMASTER** - The PropertyMASTER template is used to define the intrinsic prop-

erties or characteristics that a system or component must exhibit. This template captures non-functional requirements such as performance, scalability, usability, and quality standards [29].
Example - The system SHALL BE ABLE TO handle up to 1,000 simultaneous user logins with a response time of less than 1 second

b) **EnvironmentMASTER** - The EnvironmentMASTER template defines requirements related to the system's operational environment [29]. These requirements can include compatibility with certain hardware, software, or network conditions.
Example - The mobile banking application SHALL be designed in a way that it can be operated on both Android and iOS devices with a minimum OS version of Android 10 and iOS 13

c) **ProcessMASTER** - The ProcessMASTER template focuses on the processes or procedures the system, or its users must follow [29]. This template is particularly useful for describing operational requirements, such as workflows, maintenance procedures, and interactions with other systems.
Example - The user SHOULD confirm the transaction using a two-factor authentication code

Chapter 3

Methodology

This chapter offers an overview of the approach employed in this thesis. The primary objective of the methodology is to develop a comprehensive and systematic framework to address fairness in software systems, starting from understanding the literature to defining practical fairness-related requirements. The methodology involves 5 phases, as shown in Figure 6, these include systematically reviewing existing literature, creating a glossary of fairness terms, developing a fairness conceptual model, designing a fairness template, and ultimately applying and validating the template in real-life situations. Every phase in the process is built on the results of the previous step, resulting in a logical flow towards meeting fairness objectives. Each of these phases is explained in detail in the following sections.

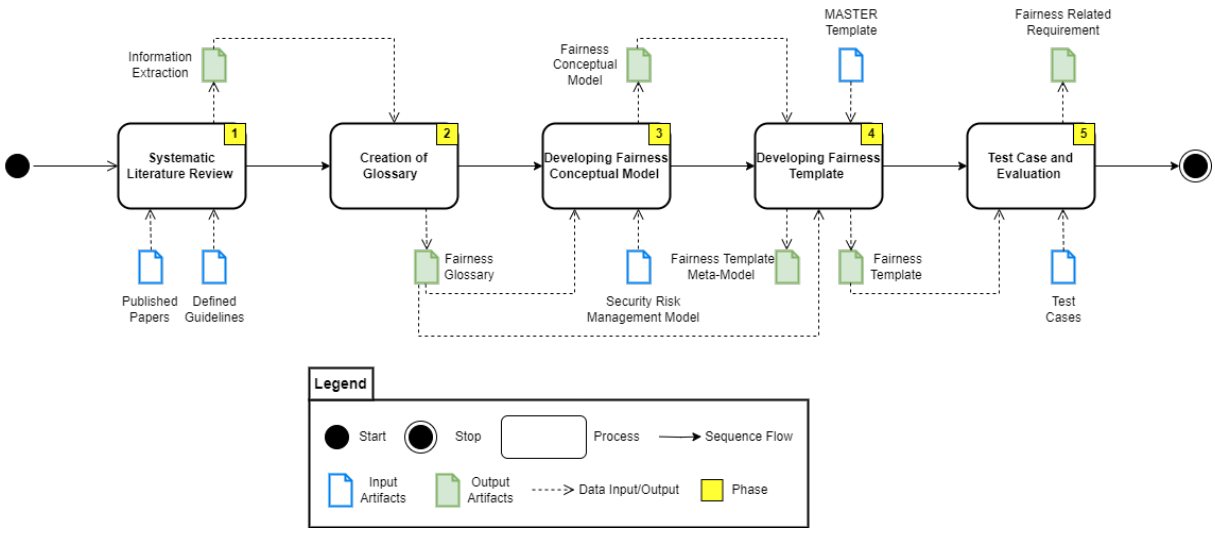

Figure 6: Methodology

3.1 Inputs

Defining fairness-related requirements in this thesis is supported by a set of well-defined inputs, each playing a unique role in the process. The main inputs include:

- **Published Papers** - These form the foundational knowledge base and provide insights into the current understanding of fairness, bias, and discrimination within software systems.

- **Defined Guidelines** - A structured set of guidelines to systematically conduct a literature review and guide the overall research process. These guidelines ensure that the review is comprehensive and that the information extracted is relevant.
- **Security Risk Management Model** - The established security risk management model serves as a framework that inspires the structure of the fairness conceptual model.
- **MASTER Template** - This requirement template serves as the basis for the development of the fairness template.
- **Test Cases** - Test cases, such as healthcare and loan approval systems, provide practical scenarios for applying and validating the fairness template.

3.2 Outputs

Although the ultimate output is to define fairness-related requirements, these intermediate outputs play a crucial role in moving from theoretical research to practical application. The key outputs are:

- **Information Extraction** - The output from the systematic literature review, guided by defined guidelines, is the extracted information. It offers an overview of the relevant paper's contributions.
- **Glossary** - The glossary consolidates key concepts and terms related to fairness, ensuring consistency and clarity in the definition of fairness requirements.
- **Fairness Conceptual Model** - This model captures relationships between various fairness elements and systematically manages fairness risks across the software development life-cycle.
- **Fairness Template** - The fairness template enables requirements engineers to define fairness-related requirements in a structured manner.
- **Fairness Template Meta-Model** - It provides a low-level, detailed representation of the fairness template.
- **Fairness-Related Requirements** - The ultimate output is the specification of fairness-related requirements using the fairness template.

3.3 Phase 1: Systematic Literature Review

The first phase of the methodology is conducting a Systematic Literature Review (SLR). The input for this step is the set of published research papers and the guidelines for conducting an SLR. These guidelines are defined based on the established guidelines outlined in "Guidelines for Performing Systematic Literature Reviews in Software Engineering" [48]. These guidelines provide a comprehensive method to develop a clear protocol, perform a comprehensive search, and systematically evaluate the quality of each primary study. The SLR focuses on RQ1, which seeks to outline fundamental concepts essential for defining fairness-related requirements in software. The aim of the SLR is to extract useful information from the literature, ensuring that the subsequent steps are grounded in the current state of the art. The SLR synthesizes the findings to provide an insightful overview of existing research in the form of an information extraction template, which forms the foundation for subsequent steps in the methodology.

3.4 Phase 2: Creation of Glossary

With the information extracted from the systematic literature review, the next step involves the creation of a fairness glossary. The purpose of this glossary is to highlight recurring themes, collect, and define all fairness-related terms, ensuring a common understanding between different stakeholders. This phase also addresses RQ1. The glossary will serve as a reference, helping ensure consistency when defining fairness requirements. The input for this stage is the extracted information from the SLR, and the output is a comprehensive fairness glossary that will be used to inform the next phase of developing the conceptual model.

3.5 Phase 3: Developing Fairness Conceptual Model

The fairness conceptual model is the third phase in the process, and it represents a structured framework for managing fairness-related risks in software systems. The inputs for this phase are the fairness glossary and the security risk management model showcased in "An Integrated Conceptual Model for Information System Security Risk Management Supported by Enterprise Architecture Management," [56] which presents a systematic approach for handling security risks. This phase addresses RQ2, by developing a conceptual model that organizes fairness concepts and identifies how they interact with each other, ensuring that fairness is systematically managed across the software development lifecycle. The primary focus of the security model was to protect information systems from threats and vulnerabilities that could result in security breaches. Similarly, the focus of the fairness conceptual model is to protect software systems from bias and unfair practices that may result in discriminatory results. The output of this phase is the fairness conceptual model, which serves as the blueprint for developing the fairness template.

3.6 Phase 4: Developing Fairness Template

One of the most important phases in integrating fairness into software requirements is the establishment of the fairness template. The fairness template developed in this master thesis is inspired by the functional MASTER template, a well-known requirements engineering template created by SOPHIST [71]. The inputs for this phase are the fairness conceptual model and the MASTER template, which offer an organized approach to document system requirements ensuring that all essential facets of software development are thoroughly covered. The goal of the fairness template is to provide a comprehensive yet flexible structure for defining fairness requirements. This phase tackles RQ3. The output of this phase is the fairness template and the meta-model of the fairness template, which is ready for practical application.

3.7 Phase 5: Test Case and Evaluation

The final phase of the methodology is the test case and evaluation, which tests the feasibility of the fairness template in real-world scenarios. The input for this phase includes the fairness template and specific test cases from domains such as healthcare insurance and loan approval systems. In this phase, the fairness template is applied to define fairness-related requirements in these systems. This phase addresses RQ4. The evaluation helps determine whether the

template is effective in capturing fairness concerns and addressing biases in software decision-making processes. The output of this phase is a set of fairness-related requirements tailored to the specific test case, which demonstrates the practical application and effectiveness of the fairness template.

Chapter 4

Systematic Literature Review

This chapter builds upon the methodological approach described earlier by presenting the detailed steps involved in conducting a literature review. In the following sections, the search strategy, including keywords, search phrases, inclusion/exclusion criteria, and sources, will be discussed, followed by an overview of the study selection process. Through this SLR, insights were gained regarding fairness definitions and concepts, which serve as the foundation for developing fairness-related requirements in software systems. Supporting documents and artifacts relevant to this chapter can be found in Appendix B: Thesis Documenation

4.1 Search strategy

This section outlines the systematic approach to identify pertinent studies for the literature review. It contains a thorough description of the keywords and search terms used, the criteria for including or excluding studies, and the sources used.

4.1.1 Keywords

The primary goal is to find pertinent studies that explore the intersection between fairness and software. A key to this undertaking was establishing a robust set of keywords that would guide the search process by finding relevant scientific publications and reliable sources.

The following set of keywords were found to be highly relevant to the research objectives, namely *Software*, *Fairness*, *Discrimination*, *AI*, and *Requirements*. These keywords align with the main objective of the thesis, which is to ensure that fairness can be embedded into software systems from the outset. *Fairness* and *Discrimination* are the cornerstones of this thesis, with the aim of ensuring equitable outcomes in software systems. *Requirements* address the SDLC phase where fairness considerations should ideally be incorporated. *Software* and *AI* are the technical domains where fairness is becoming a pressing issue, especially with the growing reliance on decision-making systems.

4.1.2 Search Phrase

A specific search phrase is used to find pertinent scholarly articles from reliable journal websites. This phrase is designed to maximize the efficiency of finding relevant articles while aligning with the core focus of the research. The phrase employed is:

(("Abstract":Software) OR ("Abstract":AI) OR ("Abstract":Requirements)) AND (("Document Title":Fair*) OR ("Document Title":Discriminat*))

The use of *Fair* and *Discriminate* in the title is due to their importance as core ideas in this thesis. This also ensures that all the retrieved papers specifically address fairness and discrimination as their primary topics. The terms *Software*, *AI*, and *Requirements* are included in the abstracts of the paper since they are very relevant to the goals of the research. It is preferable to search these terms within the abstract rather than the body of the document because it leads to an excessive number of articles, many of which may just briefly touch on the pertinent subjects, thus adding to the review burden. Thereby, the search phrase was meticulously designed to strike a balance between breadth and focus, ensuring the retrieval of highly relevant documents while avoiding an overwhelming volume of irrelevant articles.

4.1.3 Inclusion/Exclusion Criteria

The inclusion and exclusion criteria for this SLR were carefully designed to ensure that the reviewed literature closely aligns with RQ1 and the main objective of this thesis. To establish these criteria, several important factors were considered, like the research scope, relevance, quality of sources, and recent advancements in the area of fairness in software development. The intention was to exclude any material that did not contribute meaningfully to the study and concentrate on works that provide theoretical and practical insights. Below is a thorough description of each criterion.

Inclusion Criteria

- Include studies that focus on fairness or discrimination in the context of software development.
- Include academic papers and publications available in reputable peer-reviewed journals or conference proceedings.
- Include studies published within the last 10 years (2014 to present) to ensure the inclusion of recent and relevant literature.
- Include studies published in English to ensure the ability to comprehend and extract relevant information.

Exclusion Criteria

- Exclude studies focusing solely on general ethics or fairness in fields unrelated to software development.
- Exclude grey literature sources such as unpublished reports, theses, dissertations, and non-peer-reviewed materials that may lack academic rigor.
- Exclude studies published prior to 2014 to prevent obsolete concepts.
- Exclude duplicate studies or publications that present the same information, results. This prevents repetition and guarantees that the literature review contains distinct contributions.

4.1.4 Sources

The sources of literature are important to ensure the reliability and academic rigor of the research. The primary sources selected are *IEEE*, *ACM*, and *Springer*. In contrast, other platforms, such as arXiv, Elsevier, and Google Scholar, along with additional similar sources, have been deliberately excluded due to various limitations.

IEEE, ACM, and Springer platforms were chosen due to their robust peer review processes and specialization in technology and engineering. These publications cover a wide range of sub-disciplines related to the thesis topic, providing access to both groundbreaking studies and novel insights. On the other hand, arXiv, Google Scholar and other such platforms do not require peer review thereby jeopardizing the creditability of the research. Similarly, Elsevier was not used as a source for this thesis mainly because of restricted availability. Some of these platforms may focus on topics beyond engineering or technology, and they aggregate content from a wide range of sources, which may include non-academic publications, resulting in inconsistent quality. This thesis keeps its focus on reputable, pertinent, and current research by avoiding these less trustworthy sources.

4.2 Study Selection

In the initial stages of identifying relevant literature for the thesis, a broad search across three major academic sources IEEE, Springer, and ACM was scrutinized using the previously designed search string. The initial search yielded a substantial number of papers.

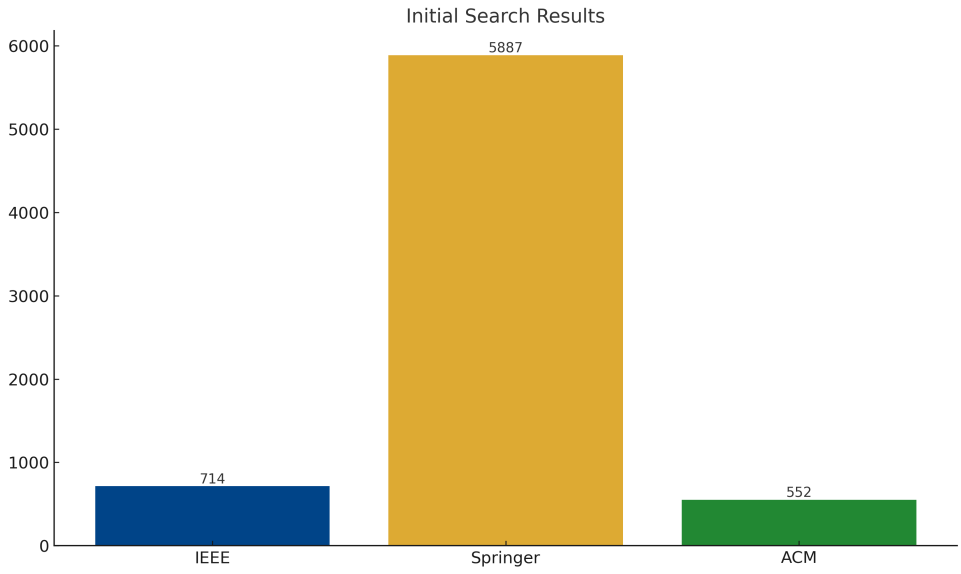

Figure 7: Initial Search Results

Figure 7 displays the initial number of papers found across three major academic databases: IEEE, ACM, and Springer. The search results yielded 714 papers from IEEE, 552 papers from ACM, and a significantly higher number of 5887 papers from Springer. The search string used was broad, covering terms related to software, artificial intelligence, and requirements with a focus on fairness and discrimination in their document titles or abstracts. This graph is instrumental in highlighting the initial volume of literature available on the topic. It shows that Springer has a vast repository compared to IEEE and ACM, which may indicate a broader

scope or more extensive indexing within this database. Understanding the initial search results helps in setting expectations for the subsequent filtering process and gives a baseline of the sheer volume of potentially relevant papers before applying any specific criteria to narrow down the focus.

To narrow down the search results, the following filters were applied to each source:

IEEE - "Conferences", "Journals", "Machine Learning", "Machine Learning Models" and "Algorithmic Fairness"
ACM - "Research article"
Springer - "Computer Science", "Artificial Intelligence", "Ethics"

These filters made it easier to focus the search on articles that directly addressed the issue of fairness in algorithmic and machine-learning contexts. These filters were intended to collect articles that addressed moral issues in computer science and artificial intelligence, particularly those pertaining to discrimination and fairness. Following the application of these filters, the search results were reduced drastically.

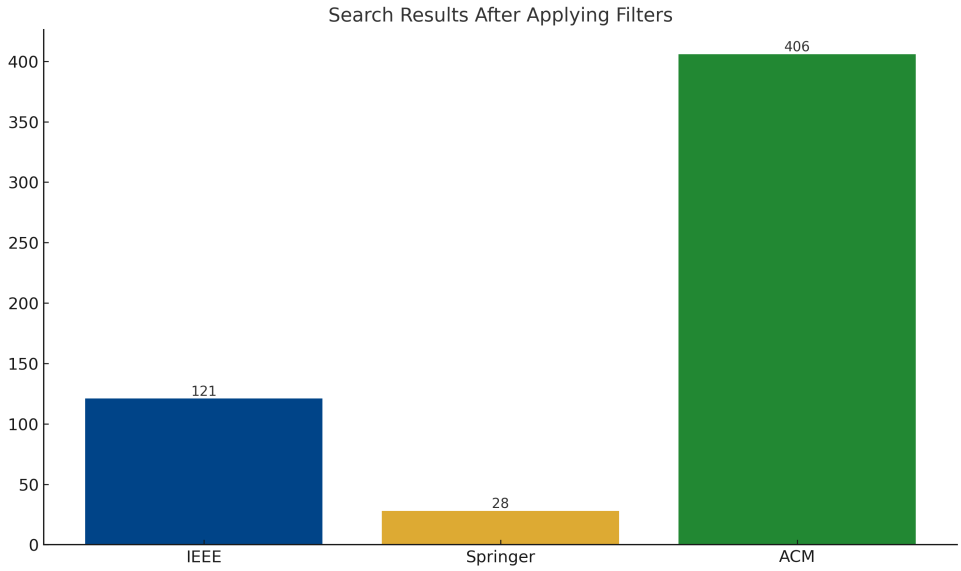

Figure 8: Search Results after applying filters

Figure 8 illustrates the number of papers remaining after applying specific filters to the initial search results. These reduced findings represented a more manageable set of papers that were specifically relevant to the research topic. The filtered search results yielded 121 papers from IEEE, 406 papers from ACM, and 28 papers from Springer. This graph is useful for demonstrating the impact of applying targeted filters to the search results. This step is crucial for refining the literature to focus on studies directly addressing fairness and discrimination in the context of software and Artificial Intelligence (AI), thus removing papers that are not pertinent to the research focus. The reduction highlights the specificity and relevance brought by applying these filters.

The next step involved meticulously going through the abstract of each paper to determine whether they address fairness, discrimination, or their related concepts in the context of software, in accordance with the inclusion and exclusion criteria. The majority of these studies dealt with fairness pertaining to computer networks or fair resource allocation in machine learning models, which is related to fairness but not in line with the study focus on the ethical and legal elements of fairness in software systems and artificial intelligence.

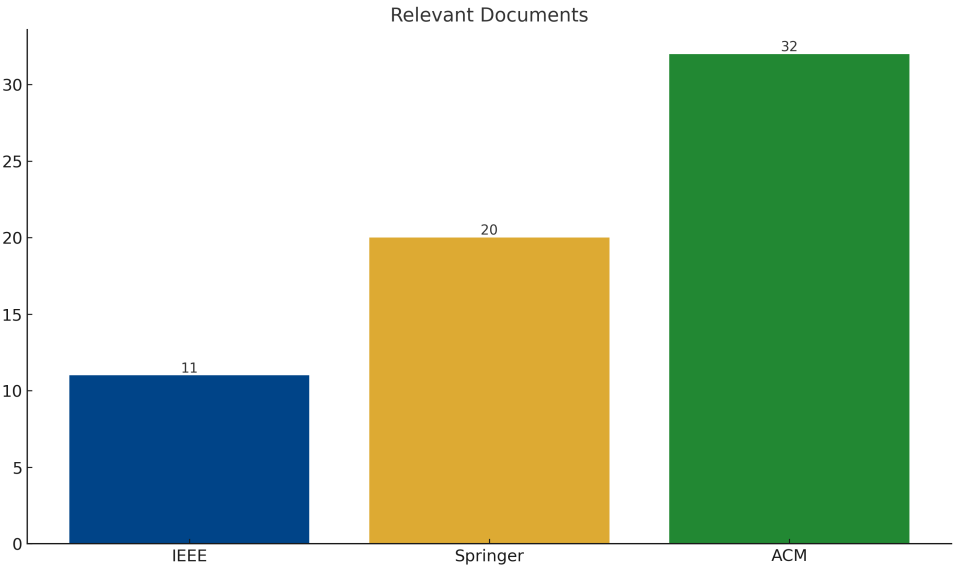

Figure 9: Relevant Papers

Figure 9 presents the final count of relevant articles after a meticulous review of the abstracts and the entire paper in some cases, to ensure they address fairness, discrimination, or related concepts in software systems and artificial intelligence. The final relevant articles were 11 from IEEE (17.5%), 32 from ACM (50.8%), and 20 from Springer (31.7%). This graph showcases the culmination of the search and filtering process, representing the core set of papers that will be considered for the thesis. It provides a clear picture of how many relevant studies each database contributes after applying rigorous selection criteria. It also helps to quickly grasp which databases are the most fruitful sources for literature on fairness and discrimination in software and AI.

Chapter 5

Information Extraction and Glossary Creation

This chapter focuses on the methodology used to systematically extract relevant data from the literature and the development of a comprehensive glossary of fairness-related concepts. The first section offers an overview of the information extraction process, discussing the rationale behind selecting specific attributes for information extraction and presenting the key findings from this process. The second section introduces the glossary which details information gathered to define fairness-related concepts and the components it encompasses. Supporting documents and artifacts relevant to this chapter can be found in Appendix B: Thesis Documenation

5.1 Information Extraction

A structured template comprising 19 attributes, each intended to capture a distinct aspect of fairness, was developed to aid this process. This structured approach ensures that information is gathered systematically, offering a concise, clear, and coherent overview of each paper's contributions. The subsections will explain the reason for selecting these attributes and will be followed by the findings from the information extraction

5.1.1 Rationale for Attribute Selection

Each attribute of the information extraction template has a distinct purpose in understanding fairness. Table 1 offers a structured overview of the selected attributes and their corresponding justification for selection, guiding the extraction of relevant information from the literature. Table 1 contains various attributes that span different dimensions of fairness, and organizing the information in this way serves as a comprehensive guide for categorizing and analyzing fairness-related aspects in different domains.

The attributes in Table 1 are not only designed to represent broad ideas of fairness but also to explore the subtleties of how fairness is implemented in practice. For instance, incorporating the *fairness measure* can aid in determining how fairness is evaluated in different studies, while *protected attributes* and *subjects of fairness* ensure that we pay attention to the specific groups impacted by fairness considerations.

Attribute	Reason
Fairness Mentioned	Indicates whether fairness is discussed explicitly or implicitly, aiding in determining its significance within the paper
Authors' Understanding of Fairness	Describes the way fairness is interpreted in every paper. Given that fairness can be interpreted in various ways, this attribute aids in categorizing the authors' interpretation of fairness
Motivation for Fairness	Identifies the reasons behind addressing fairness. This helps understand the importance and relevance of fairness in different applications
Laws and Standards	Monitors whether the paper references existing legal frameworks that guide fairness interventions. This is crucial to comprehending how fairness complies with or deviates from legal requirements
Application Domain	Determines the domain in which fairness is applied. Fairness concerns can differ significantly across applications, and this attribute aids in discovering problems unique to a given domain
Fairness Measure	Describes the fairness metrics referenced in the paper, helping comprehend how fairness is quantified
Protected Attributes	Specifies the distinct attributes (e.g., race, gender, age) that are the main focus of the paper. This shows which groups are being included in assessments of fairness
Subjects of Fairness	Describes the individuals, organizations, or systems impacted by unfairness, offering perspective on how fairness is perceived at an individual level
Type of Discrimination	Identifies if the paper emphasizes direct discrimination or indirect discrimination
Mention of Proxy	Determines if proxy variables are utilized instead of protected attributes
Measurement of Proxy	Determines whether specific tools or techniques are used to address proxies
Type of Work	Classifies the paper based on its contribution (e.g., Detect, Evaluate, Monitor, Mitigate fairness issues)
Phase of Work	Monitors the stage in the system lifecycle where fairness is considered. This is essential for determining when to implement fairness
Type of Paper	Classifies papers based on their type (e.g., empirical study, technical paper, theoretical work, review)
Threats to Fairness	Points out any potential dangers or risks mentioned by the authors regarding upholding fairness. For example, issues like model drift or biases introduced during training could compromise fairness
Fairness Trade-offs	Determines if there are any compromises made between fairness and other objectives
Quantitative Metrics	Documents the quantitative metrics used to measure fairness, like demographic parity, precision, recall, etc.
Qualitative Metrics	Captures any qualitative assessments of fairness, such as interviews, case studies, or theoretical discussions
Other Terms	Gathers new or upcoming terms connected to fairness, like anti-protected

Table 1: Reason for Selecting Attributes

Furthermore, attributes such as *type of discrimination*, *mention of proxy*, and *measurement of proxy* focus on identifying indirect or subtle forms of bias, highlighting areas where fairness may be compromised through the use of proxies or unintended consequences.

The *type of work*, *phase of work*, and *type of paper* attributes are included to categorize research according to their contribution, the specific stage in the system lifecycle when fairness is considered, and the research methodology applied. This ensures that the research encompasses a broad spectrum of studies, ranging from theoretical investigations to practical assessments, and focuses on the full software development lifecycle.

Lastly, attributes like *threats to fairness*, *fairness trade-offs*, and *quantitative metrics* and *qualitative metrics* focus on the challenges and compromises that arise in achieving fairness. By tracking these factors, we can better understand the limitations, risks, and trade-offs involved in balancing fairness with other objectives.

5.1.2 Findings of Information Extraction

This section presents the key findings from the information extraction process. Related attributes have been grouped together for clarity and depth, facilitating a deeper comprehension of the data. The section intends to provide a comprehensive overview of fairness-related research by organizing the findings in this specific manner, highlighting both specific details and overarching themes.

Fairness Conceptualization and Motivation

- **Fairness Mentioned** - Fairness was explicitly referenced in about *85% of the papers*, highlighting a significant emphasis on the concept. These papers commonly address fairness as a fundamental issue in the decision-making process, especially in terms of reducing bias and achieving fair outcomes. Conversely, *15% of the papers* referred to fairness implicitly, typically addressing topics related to bias or inequality without directly using the term fairness.
- **Authors' Understanding of Fairness** - About *90% of the papers* define fairness in terms of equal treatment towards individuals or groups. *The remaining 10%* offer more complex explanations, like the importance of fairness in the decision-making process or maintaining fairness in various stages of the software development lifecycle.
  - *In 45% of papers*, the prevailing definition is that fairness involves treating individuals or groups equally regardless of protected attributes such as race or gender.
  - *30% of the papers* emphasize fairness as the absence of discrimination against specific groups.
  - *15% discuss* fairness in terms of procedural or distributive justice.
- **Motivation for Fairness** - *75% of the papers* acknowledge that fairness is often driven by the desire to mitigate bias. This bias is perceived as a risk that, if not dealt with, could reinforce systemic inequalities. *25% of the papers* are influenced by legal standards and societal pressures when discussing fairness emphasizing adherence to regulations such as GDPR and anti-discrimination laws.
- **Laws and Standards** - Only a mere *20% of the papers* explicitly mention current laws and standards regarding fairness.

Fairness Measures and Discrimination

- **Fairness Measures** - Figure 10 illustrates the distribution of fairness measures used across the reviewed literature. Statistical parity, which assesses if various groups receive similar outcomes from an algorithm, is the most prevalent metric cited in *60% of the papers*. *30% of the papers* address fairness through the lens of equality of opportunity, focusing on providing individuals from different groups with equal chances for success. Additionally, *10% of the papers* utilize more specialized measures relevant to specific domains, such as healthcare or finance. Fairness can be understood as the prevention of harm, providing equal access to resources, and ensuring fair distribution of resources in these situations.

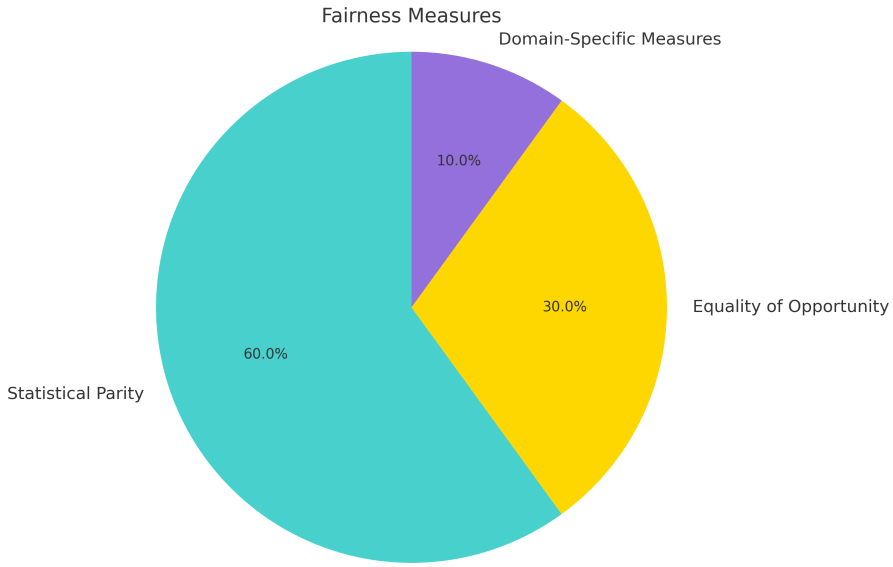

Figure 10: Distribution of Fairness Measures

- **Protected Attributes** - Age, gender, and race are the three main topics of discussion in *70% of the publications*, with gender and race being the most often mentioned protected attributes. Additionally, *20% of the papers* incorporate other attributes, such as socioeconomic status or disability, which are gaining recognition in fairness assessments.
- **Type of Discrimination** - In *40% of the papers*, the discussion revolves around direct discrimination (e.g., an algorithm that uses race as a decision criterion). With indirect discrimination accounting for *60% of the papers*, it is more frequently studied when seemingly neutral variables produce biased results. Here, the emphasis is on how variables like zip codes or educational qualifications might act as proxies for protected attributes.

Work Type and Application Domains

- **Type of Work** - Figure 11 displays the distribution of various types of work related to fairness in the reviewed literature. Almost a third of the studies (*35% of the papers*) are devoted to evaluating fairness in current systems. The evaluation process involves assessing the system's performance, while also identifying potential biases and fairness concerns. An additional *25% of the papers* focus on detecting bias, developing techniques to find biased data or unfair outcomes in models. *20% of the research* effort is focused on monitoring fairness to guarantee its consistency in all stages of the software development

lifecycle, particularly post-deployment. Lastly, *20% of the papers* offer methods for mitigating bias by aiming to actively decrease or eliminate fairness concerns during model training or data processing.

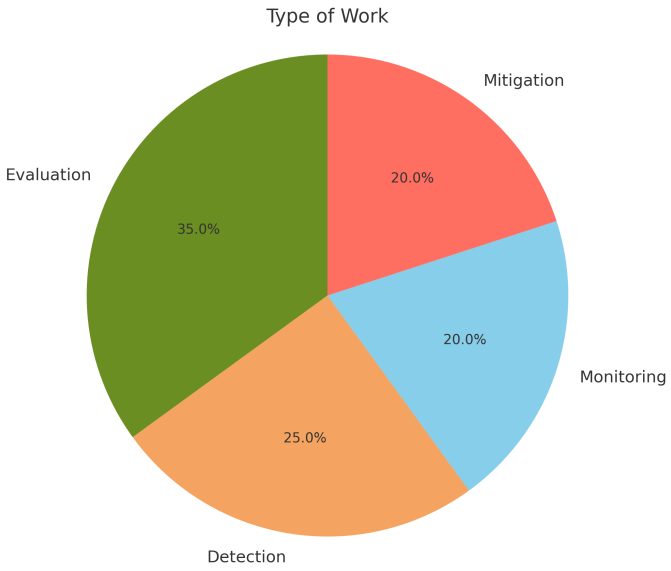

Figure 11: Distribution of Different Types of Work

- **Application Domains** - Figure 12 depicts the distribution of application domains in the reviewed literature, emphasizing which areas have the greatest focus on fairness concerns. *33.3% of the papers* are devoted to finance and banking, including topics such as credit scoring and loan approval bias. In the field of healthcare, where predictive models are used for diagnosis and treatment, *27.8% of the papers* address fairness in such scenarios. Additionally, *22.2% concentrate* on software systems, emphasizing fairness in the design and implementation of machine learning applications. *The other 16.7%* focus on government and legal systems, emphasizing the importance of fairness in decision-making algorithms, like those employed in criminal justice and public policy.

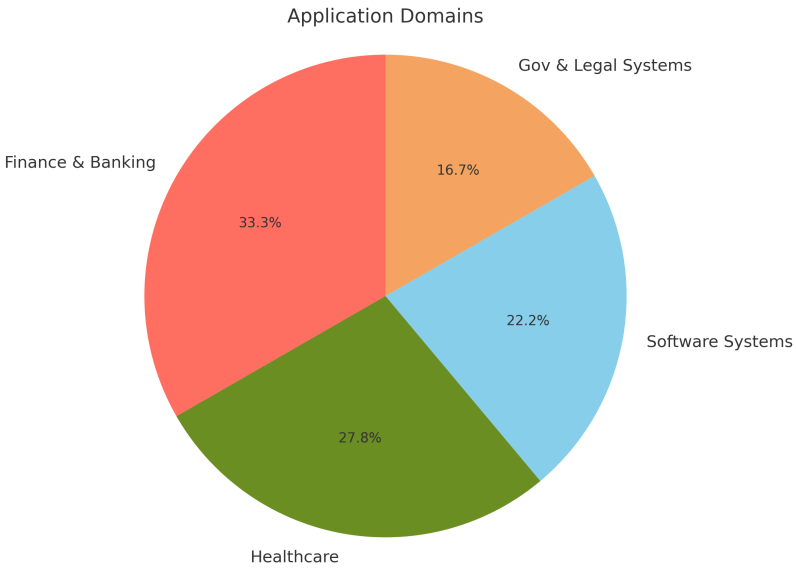

Figure 12: Distribution of Application domain

- **Type of Paper** - *50% of the papers* reviewed are empirical studies that provide practical assessments of fairness in different systems. *20% of the papers* are theoretical and offer conceptual frameworks for defining and comprehending fairness. Technical papers account for *15% of the total*, offering solutions and techniques for identifying and addressing bias. Lastly, Survey and review studies comprise *15% of the reviewed literature*, summarizing the body of research to draw attention to up-to-date trends and issues related to fairness.

Fairness Threats, Trade-offs, and Metrics

- **Threats of Violating Fairness** - *70% of the papers* point out clear risks to fairness, such as the use of proxy variables, which can lead to the creation of new types of bias or the reinforcement of preexisting ones despite mitigation efforts. *The remaining 30%* address these issues, although in a less detailed manner, with a greater emphasis on theoretical risks rather than real, observed threats.
- **Fairness Trade-offs** - Approximately *60% of the papers* examine the balance between fairness and various performance metrics like accuracy or efficiency.
  - *40% of the papers* mention that enhancing fairness can result in decreased accuracy models may lose precision when they are altered to satisfy fairness requirements.
  - The balance between minimizing bias and preserving data utility is highlighted in *30% of papers*, especially in industries like banking or healthcare where data accuracy is just as vital as fairness.
- **Qualitative Metrics** - *25% of the papers* address qualitative evaluations, such as conversations about the societal foundations of bias or fairness.

Emerging Terms and Concepts

- **Other Terms** - Throughout the literature, several novel and developing terminology pertaining to fairness were found. With *20% of papers* mentioning them, anti-classification and classification parity were the most common topics. These words refer to techniques that stop algorithms from making decisions based on protected features. In addition, *15% of papers* mentioned ideas like distributive justice and procedural justice, giving talks of fairness a deeper ethical and philosophical dimension.

5.2 Glossary Creation

This section aims to provide a clear understanding of key terms that serve as a guide to navigating complex discussions surrounding fairness in software systems. The subsequent subsection will provide an overview of the glossary template and the glossary as a whole.

5.2.1 Glossary Template

The glossary is structured to offer a thorough summary of important fairness-related terms. For each term, the following information is included:

- **Definition** - Brief description of the term.

- **Alternate Terms / Alias** - Other names or phrases that are commonly used to refer to the same concept.
- **Source** - A citation from scholarly articles that discusses or introduces the term.
- **Example** - A situation demonstrating how the term is applied in real life.

5.2.2 Overview of Glossary

The glossary created in this thesis, as represented in Table 4 in Appendix A: Glossary, serves as a solid foundation for understanding various fairness-related concepts critical to both the academic and practical contexts of software systems. The glossary acts as a common language to ensure that all stakeholders are aligned when discussing fairness requirements, methodologies, and outcomes.

In creating a comprehensive glossary of fairness-related concepts, it became necessary to group the terms into distinct categories that address various facets of fairness in software systems. This structured approach allows for a better understanding of how the various components collaborate to ensure fairness in decision-making processes. The glossary is categorized based on different aspects of fairness in software systems. Organizing the glossary in these categories allows for better exploration of various fairness concepts, and methods, ensuring that every aspect of fairness is thoroughly covered during the software development process.

The different categorizations and an outline of concepts included in each category are mentioned below:

- *Fairness Foundations* - This category includes concepts like fairness goals, fairness requirements, fairness techniques, fairness audits, fairness constraints, and fairness measures.
- *Types of Fairness* - This category includes group fairness, statistical parity, equal opportunity, equalized odds, predictive equality, predictive parity, conditional demographic parity, treatment equality, calibration, individual fairness, fairness through awareness, fairness under unawareness, and counterfactual fairness.
- *Attribute Considerations* - This category encompasses protected attributes, anti-protected attributes, proxy discrimination, and exploratory variables.
- *Risk Management Concepts* - This group covers vulnerabilities, threats, impact, and risk.
- *Stakeholder and Data Subject Considerations* - This classification includes stakeholders and data subjects.
- *Advanced Fairness Concepts* - This category includes subgroup fairness, intersectional fairness, temporal fairness, robust fairness, and causal fairness.

Fairness Foundations

This category is essential for ensuring fairness in software systems. They establish the basis for defining and upholding fairness. It consists of the overall *fairness goals*, which are the principles guiding fairness, and *fairness requirements*, which are actionable measures stemming from these objectives. To achieve these requirements, various *fairness techniques* or controls are implemented. To ensure ongoing compliance with fairness requirements, systems require *fairness audits* and *fairness measures* to consistently assess and verify fairness through evaluations

and metrics. *Fairness constraints* place certain restrictions on how decisions should be made to ensure equitable outcomes. This category serves as the backbone of any fairness initiative, ensuring that fairness is not only defined but also implemented, monitored, and maintained.

In this thesis, these concepts are vital for establishing a systematic approach to fairness, guiding the development of fairness requirements from a conceptual level to practical implementation.

Fairness Measures

These fairness measures are grouped together because they represent the diversity of fairness definitions and approaches. They provide the flexibility to customize fairness measures to the specific needs of a system by defining what fairness means in various contexts. Developers can ensure fairness in several ways by taking these different measures into account, whether it involves equalizing results among different groups or making sure that similar individuals receive equal treatment.

In this thesis, these fairness measures help shape the fairness requirements in a way that best suits the specific context in which the software is being developed.

Atrribute Considerations

This category focuses on attributes used in decision-making procedures. These terms are categorized together as they focus on the proper handling of data to avoid discrimination. Understanding the significance of protected and non-protected attributes is crucial for creating unbiased systems and identifying possible proxy discrimination is essential for reducing indirect bias.

In this thesis, accurately classifying these attributes is crucial for establishing fair requirements that prevent discrimination, both direct and indirect, and ensuing equitable decision-making processes.

Risk Management Concepts

These terms are categorized together because they represent the systemic risks to fairness within a software system. Just as in security management, identifying and mitigating these risks is essential for ensuring that a system upholds fairness. This risk-based approach to fairness ensures that potential biases are detected and addressed early in the system development process.

In this thesis, risk management concepts inform fairness strategies by identifying potential vulnerabilities in the system that could jeopardize fairness.

Stakeholder and Data Subject Considerations

This category includes the human elements of fairness. These terms are grouped due to their emphasis on the practical consequences of fairness. While fairness can be discussed in technical terms, the ultimate goal is to ensure that individuals are treated fairly and that stakeholders understand how decisions are made. Engaging stakeholders and respecting the rights of data subjects are crucial for building trust and accountability.

In this thesis, engaging stakeholders and examining the impacts on data subjects ensures that fairness is seen as more than just a technical concern, but also a societal and ethical one.

Advanced Fairness Concepts

These terms are grouped because they represent more nuanced and specialized fairness measures. These concepts are essential for ensuring fairness in complex systems that may have overlapping or evolving fairness requirements. They offer more sophisticated tools for handling fairness, especially in dynamic environments where fairness must be maintained over time or across multiple dimensions.

Chapter 6

Conceptual Model

This chapter provides a detailed explanation of the development of the fairness conceptual model. It begins by discussing the inspiration drawn from an existing security risk management model [56], which helped guide the structure and approach of the fairness conceptual model. The chapter then introduces newly added components that were specifically designed to address fairness-related concerns in software systems. Finally, an overview of the entire fairness conceptual model is presented, highlighting its role in systematically managing and mitigating fairness risks throughout the SDLC.

6.1 Inspiration from the Original Model

While developing a conceptual model for fairness, several core components from the original security framework were recognized as important to be included. While the new model emphasizes fairness, these components have been repurposed to offer a systematic method for handling and reducing fairness-related risks. Each of these components has been incorporated into the fairness conceptual model due to their wide applicability and their effectiveness in addressing system needs, risk management, and implementation strategies. Table 2 outlines the components being reused, along with a general description, followed by a thorough explanation of each component and their reason for inclusion.

Requirement - This component was reused because of its ability to influence system design and behavior. Requirements include prerequisites that a system needs to fulfill to accomplish its overall goals. In the original model, requirements were used to outline the system's security objective whereas, in the new fairness model, it defines the guidelines the system must meet to ensure fair outcomes.

Risk - In the original model, risk points out possible events or conditions that may jeopardize system security. In the new fairness model, this concept is adapted to highlight the possibility of unfair outcomes. By reusing this component, the fairness model can recognize, evaluate, and control fairness-related risks like how the security model addresses security risks.

Risk Treatment - In both the original security model and the fairness model, the risk treatment component encompasses the strategies and actions taken to mitigate identified risks. In the original model, this component was used to preserve the system security by including controls and safeguards. Likewise, the new fairness model includes methods to reduce fairness risks, like bias reduction techniques or modifications to decision-making procedures.

Component	Description
Requirement	The overall requirements and conditions necessary to accomplish goals within a system
Risk	Possible scenarios or situations that may result in negative consequences in a system
Risk Treatment	This involves strategies and actions taken to mitigate identified risks
Impact	Explains the outcomes that occur when detected risks become a reality in the system
Event	Occurrences that can initiate risks
Vulnerability	Indicates flaws in the system that could be exploited and have unfavorable effects
Threat	Actions or circumstances that have the potential to take advantage of system
Technique	Methods or tools employed in a system to carry out specific controls, requirements, or measures

Table 2: Reused Components from Security Model

Impact - The concept of impact from the original model is directly applicable to the fairness model. The addition of this component to both models makes it easier to assess the severity of risks and helps prioritize risk treatment strategies.

Event - In the original model, an event refers to an incident that may result in security risk, such as a data breach. Similarly, the fairness model refers to an incident that may result in fairness risks, such as introducing biased data. By keeping this component, the fairness model emphasizes the need to pinpoint particular incidents that may lead to fairness concerns.

Vulnerability - In the security model, vulnerability refers to the systemic flaws that may be exploited by threats. This idea is repurposed in the fairness model, where these flaws may result in unfair outcomes. Incorporating this component in the fairness model proves the need to identify and rectify vulnerabilities within the system, to prevent their exploitation and maintain fairness.

Threat - In the original model, a threat is an activity that has the potential to compromise system security by exploiting vulnerabilities. This idea easily applies to the fairness model, where threat includes elements that can take advantage of vulnerabilities to create unfair results. By keeping this element, the model acknowledges the significance of identifying risks to prevent unfairness in software systems.

Technique - In the original model, technique refers to the methods employed in order to achieve system security. In the fairness model, this idea is applied again to indicate techniques or tools used to enforce fairness. The inclusion of Technique in each model emphasizes the necessity of taking concrete actions to accomplish system goals, whether security or fairness-related.

6.2 Newly Added Components for Fairness

The fairness conceptual model introduces several new components to address the specific challenge of ensuring fairness in software systems. Although the original security model offered

a strong foundation, the additional components now include considerations for fairness, and the wider scope of individual rights, and the overall integrity of the system. Table 3 below outlines the newly added components, along with a general description, followed by a thorough explanation of each component and their reason for inclusion.

Component	Description
Fairness Requirement	Specific criteria that the system must meet to ensure its fairness
Fairness Technique	Specific methods designed to enhance and implement fairness within systems
Fairness Measures	Measures used to assess fairness in software systems
Audits	Regular evaluations to ensure compliance with fairness requirements
Data Subject	Individuals whose data is processed and analyzed by the system
Protected Characteristics	Attributes of individuals that are legally protected from discrimination, such as race, gender, and age
Anti-Protected Characteristics	Attributes that may be used for making decisions
Direct Risk	Risks that stem directly from the use of protected attributes in the decision-making process
Indirect Risks	Risks that arise not from the explicit use of protected attributes, but rather through the use of proxy variables that correlate with these attributes
Proxies	Variables that indirectly represent protected characteristics and can introduce bias
Security Technique	Specific methods designed to enhance and implement fairness within systems
Attackers	Entities that attempt to exploit vulnerabilities in the system to compromise its security and fairness
Security requirements	Conditions that the system must meet to ensure its security
Threat Agent	Actions that may attempt to harm the system by exploiting its vulnerabilities
Attack Methods	Techniques used by attackers to compromise the system
Decision Making Agent	Refers to the system that, if flawed, can lead to the exploitation of vulnerabilities and result in biased or unfair outcomes
Security Specific	Refers to vulnerabilities specifically related to security aspects of the system
Bias Specific	Refers to vulnerabilities that are directly associated with biases within the system

Table 3: Newly Added Components to Fairness Model

Fairness Requirement - This component enables developers to set specific and actionable goals to drive the system's design and development process. These requirements are also essential for evaluating if the system is meeting its fairness objectives.

Fairness Technique - This component encompasses various methods and strategies to encourage fairness within the system. This is a crucial aspect because it specifically deals with modifying or enhancing the system to guarantee equitable results. Through the use of fairness

techniques, the model includes effective solutions to reduce biases and discriminatory practices.

Fairness Measures - This component offers a framework for evaluating whether the system meets its fairness requirements. These measures may either be *qualitative* or *quantitative*. Fairness measures ensure that the objective is measurable and enable the model to evaluate the fairness of the system by comparing its performance against predetermined benchmarks.

Audits - Audits are a continuous process that helps identify biases and areas that need to be improved, to ensure the system is in line with the fairness objectives. This confirms the notion that fairness is a continuous process that calls for constant monitoring and modification.

Data Subject - This component is used to identify individuals whose data is being processed by the system. This acknowledges that the system's decisions have adverse effects on the individuals who use it and therefore, serve as a foundation for safeguarding people's rights.

Protected Characteristics - This component represents attributes of individuals that are legally protected from discrimination, such as race, gender, and age. It is crucial to determine what criteria the system should specifically avoid using for decision-making.

Anti-Protected Characteristics - This component represents attributes of individuals that the system can use in the decision-making process, such as credit score and employment status.

Personal Data - This component refers to any information that can be used to identify an individual. It may either be sensitive or insensitive and has the potential to introduce biases if dealt with incorrectly.

Direct Risk - Risks that originate directly from incorporating protected attributes in decision-making. It is crucial to identify and mitigate these risks immediately because their presence confirms the use of sensitive attributes leading to unbiased outcomes. This component was added to the conceptual model to highlight the potential for explicit discrimination.

Indirect Risk - These risks arise from the use of proxies rather than the direct use of protected attributes. Indirect risks are more subtle and trickier to uncover as they unintentionally cause discrimination by depending on an attribute that acts as a stand-in for the protected attribute. This inclusion assists in identifying and mitigating discriminatory practices that could go undetected if just direct risks were taken into account.

Proxies - Proxies are variables that can induce bias into decision-making processes and serve as an indirect representation of protected attributes. By incorporating proxies, the model highlights the importance of identifying and dealing with this bias.

Security Technique - This component encompasses various methods and strategies to protect the system against possible threats and vulnerabilities. This is a crucial aspect because it protects the system from attacks from an external agent, who may exploit system weaknesses.

Attacker - This component represents entities that may attempt to exploit vulnerabilities in the system to compromise its fairness. This component is present as a proactive measure to safeguard the system from intentional external attacks that may wrongfully influence the decision-making outcomes.

Security requirement - This component outlines the prerequisites that must be fulfilled to guarantee the resilience and integrity of the system. This element was included to emphasize the need to protect the system from security risks, that can compromise its goals, just as much as ensuring fairness.

Threat Agent - A threat agent is something that could jeopardize the system. It could either be an attacker, who is a person or an outside force trying to exploit the system, or it could be the decision-making process, which occurs when the system has flaws and leads to biased decisions. This component highlights the necessity to secure the system against the dual nature of threats: external and internal.

Attack Methods - This component refers to the methods employed by attackers to breach the system's security and disrupt its operations.

Decision-Making Process - This refers to the system in charge of making decisions. If there are intrinsic flaws in this process, such as poor design, biased training data, or incorrect algorithms, it can exploit the vulnerabilities and produce unjust results. Adding this component highlights the significance of making sure algorithms and decision-making processes are devoid of biases and vulnerabilities that may jeopardize fairness.

Security Specific - This refers to vulnerabilities in the system that can be exploited to undermine its integrity and fairness. The conceptual model was extended to include security-specific vulnerabilities to show how system security and fairness are related. A system without adequate security measures is vulnerable to manipulation, leading to unjust results.

Bias Specific - This refers to vulnerabilities in the system that induce or maintain bias in the decision-making process. By incorporating this element, the model can better guide initiatives in developing fair systems by defining fairness requirements and strategies that effectively identify, reduce, and manage biases.

6.3 Overview of the Fairness Conceptual Model

Figure 13 is a graphical representation of a conceptual model, that offers a framework for understanding and maintaining fairness in software systems. It exhibits the interconnectedness of various components to establish a methodical approach to defining, assessing, and upholding fairness during the software development process. This conceptual model is carefully designed to incorporate both fairness and security-specific components, emphasizing the need to ensure a fair outcome while also upholding system integrity. In this model, the grey boxes represent components that were repurposed from the original security model and show how basic security concepts can be modified to address fairness. The white boxes represent newly added components that explicitly address issues related to fairness in software systems.

The model begins with *Requirement*, which is classified into *Fairness Requirement* and *Security Requirement*, which defines the prerequisites that the system must fulfill to accomplish its security and fairness goals. Fairness requirements specify *Fairness Measures*, which are used to evaluate compliance and serve as both qualitative and quantitative indicators. The model highlights that attaining fairness is a continuous process, in which *Audits* is an essential aspect. Audits employ fairness measures to periodically assess the system, making sure that its decisions meet the specified requirements. This feedback loop indicates that fairness is a dynamic process that calls for continuous observation and modification.

At the core of the model are *Risks*, which can either be direct or indirect, indicating the potential threats to the system's fairness and integrity. *Events* play a crucial role here. Events are incidents that have the potential to trigger these risks, such as implementing skewed data into the system, errors in the decision-making process, or having external attackers exploit vulnerabilities. The model highlights the need to identify such incidents that could raise fairness issues, facilitating better risk management and mitigation plans. The *Impact* component is strongly

connected to risks and events, reflecting the outcomes that occur when a risk becomes reality. Understanding the impact is essential for determining the severity of risk and also to prioritize risk treatment strategies. It emphasizes how fairness concerns have practical consequences and how crucial it is to address both short-term and long-term impacts.

Threats and *Threat Agents* represent the sources of risk, including external attackers or internal flaws within the decision-making process. These agents could take advantage of *Vulnerabilities* in the system, resulting in unfair results. For example, an algorithm used for making decisions could become a threat if it is flawed or poorly designed, as it may introduce biases in the training data, producing unfair outcomes. The *Attack Methods* component is linked to threat agents, indicating the various strategies that could be employed to exploit vulnerabilities within the system.

Risk Treatment is a significant component of the conceptual model, used to handle the aftermath of events that could jeopardize fairness. It serves as a framework guiding the choice of suitable approaches and strategies required to manage risks. For instance, if an event, such as the introduction of biased data, is recognized as a source of bias, risk treatment could entail changing data collection methods or retraining the algorithm with balanced data. This component again emphasizes the need for a proactive and continuous strategy to address potential bias, showing that upholding fairness is a continuous process.

Technique components include a range of tools and strategies designed to promote fairness and security within the system. This component is further classified into *Fairness Technique* and *Security Technique*. Fairness Techniques involve targeted strategies aimed at improving fairness within a system. These may include methods such as bias detection algorithms, data balancing techniques, and fairness-aware decision-making processes. Security Techniques, on the other hand, focus on safeguarding the system against external threats and vulnerabilities that could jeopardize its fairness and integrity. These techniques might include encryption, access controls, anomaly detection, or intrusion prevention systems. It indirectly supports risk treatment by reinforcing the overarching goal of reducing bias and protecting the system from potential threats.

The inclusion of *Data Subject* serves as a stark reminder that individuals are directly impacted by the decisions made by the system. This emphasizes the ethical need to take fairness and transparency seriously. The data subject has *Personal Data*, and it is linked to risks indicating that inappropriate use, storage, or analysis of personal data can lead to biases and unfair outcomes. The model underscores the significance of data protection and ethical issues by incorporating personal data, guaranteeing that decisions made by the system uphold the rights and privacy of the people they impact. Personal data is further classified into *Protected Characteristics* and *Anti-Protected Characteristics*. Anti-protected characteristics have the potential to serve as *Proxies* for protected characteristics. For instance, an individual's zip code might serve as a proxy for their race or socioeconomic status. In the model, proxies are linked to indirect risks, highlighting the subtle ways in which discrimination can infiltrate algorithms, even when protected characteristics are not explicitly utilized. The model highlights the importance of closely examining all input variables to make sure they do not unintentionally serve as stand-ins for protected attributes.

Overall, the conceptual model offers a comprehensive understanding of the complex network of components that impact fairness in software systems. In addition to identifying the components that maintain fairness, this model also shows how they are interconnected. This integrated framework shows that maintaining fairness calls for consistent, diverse actions, including thorough risk assessment, active utilization of fairness and security methods, and continuous monitoring with audits. By combining these aspects, the model acts as a roadmap for dynamically and systematically integrating fairness into software systems.

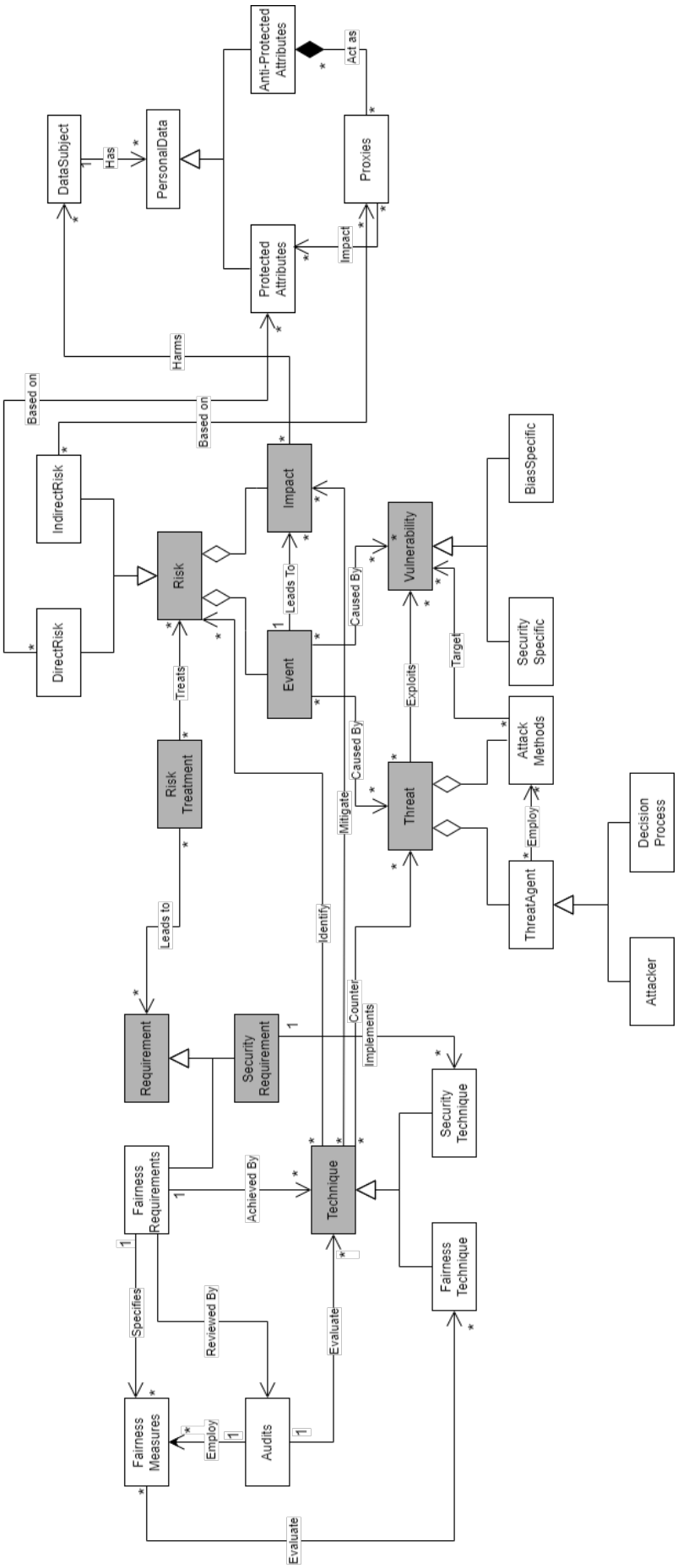

Figure 13: Conceptual Model for Fairness

Chapter 7

Fairness Template

The chapter will explore the overview of the fairness template and the meta-model of the fairness template, demonstrating how the template has been adapted to include fairness-specific elements and how the meta-model illustrates the interconnections between these components within the system.

7.1 Overview of Fairness Template

The Fairness Template, as illustrated in Figure 14, serves as a comprehensive framework for incorporating fairness requirements within software systems. Expanding upon the tenets of the functional MASTER template, it adds a new emphasis on fairness by modifying the requirement template to tackle both bias and equity. In Figure 14, the components highlighted in grey represent elements that have been reused from the functional MASTER template, reflecting the template's roots in established requirements engineering practices. Conversely, the white components in the template are newly defined and tailored to address fairness specifically.

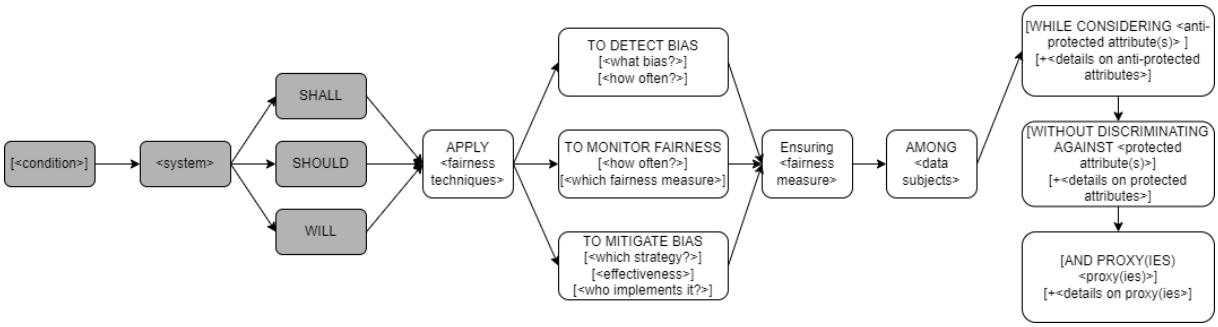

Figure 14: Fairness Template

It enables requirements engineers to specify which part of the system will enforce fairness, the data subject it will affect, what fairness techniques will be applied, and which fairness metrics will be used to evaluate the outcomes. The template also provides flexibility, allowing optional components such as conditions, protected attributes, anti-protected attributes, and proxies, which give stakeholders the ability to customize fairness requirements based on the specific context or domain. This level of customization allows the template to accommodate different fairness goals and assists non-expert requirements engineers in defining fairness requirements, ensuring that considerations of fairness are integrated seamlessly into software systems.

To illustrate the application of the fairness template, let's consider a job recruitment platform that uses an automated system to process job applications. The system needs to ensure fairness in shortlisting candidates for interviews. Here is how the template might be applied:

Example: <The job recruitment system> SHALL APPLY <adversarial debiasing techniques> TO DETECT BIAS [IF the shortlisting rate disparity between age groups exceeds 10%], ENSURING <demographic parity> AMONG <job applicants>, WITHOUT DISCRIMINATING AGAINST <age>

7.1.1 Components of the Fairness Template

The fairness template adds to the structure of the functional MASTER template. A detailed breakdown of each component is given below:

Condition - This component establishes the context in which fairness must be enforced by the system granting more precise control over when to apply fairness techniques. For instance, a condition can be *during peak hours* or *while handling loan requests*. Stakeholders can now customize the requirements for fairness to fit specific scenarios by adding conditions, which guarantees that the behavior of the system adjusts to various environments. This is an optional component.

<System> - The component recognizes the software system or module accountable for enforcing the fairness requirements. It specifies which section of the entire system is responsible for implementing fairness methods, whether it is the entire software system, a specific algorithm, or a particular decision-making component. With this classification, the fairness requirement's scope is made clearer, helping define where the fairness mechanisms should be incorporated into the software system. For instance, a system could be *the loan approval system* or *the recommendation engine*.

Shall / Should / Will - These terms define the obligation or commitment level of the fairness requirement within the template. *Shall* indicates a mandatory requirement that the system must fulfill. *Should* suggest a recommendation rather than a strict requirement. *Will* expresses intent or future action.

Apply<Fairness Techniques> - The component specifies the specific methods or strategies the system will use to uphold fairness. For instance, these may include *reweighting techniques* or *disparate impact analysis*.

To Detect Bias / To Monitor Fairness / To Mitigate Bias - This part of the template outlines the specific goals of the fairness techniques being applied. *To identify bias* indicates that the system must detect any bias instances or patterns in its decision-making processes. *To monitor fairness* involves continuously evaluating the fairness of the system as time progresses. *To mitigate bias* outlines strategies for reducing or eliminating identified biases.

Ensuring<Fairness Measure> - The component defines the metrics and measurements that will be used to assess the fairness of the system. They may either be quantitative or qualitative, based on the system's context and the specific attributes being protected. For instance, these may include *statistical parity*, *equal opportunity*, and *predictive parity*. By defining the fairness metric, this element establishes a standard by which the system's actions will be judged, giving a clear signal of whether fairness goals are being achieved.

Among<Data Subjects> - This component identifies the individuals or groups impacted by the fairness requirements, such as *users of a loan application system*, *students in an educational platform*,

or *employees in an HR software system*. The template makes sure that the fairness techniques are personalized for the specific population the system serves, by pinpointing the data subjects.

While Considering <Anti-Protected Attributes> - This component indicates the characteristics that the system is allowed to use in its decision-making process. A few examples of such characteristics are *credit score*, *employment status*, or *educational background*. This component is optional as not all fairness requirements need to specify anti-protected attributes.

Without Discriminating Against<Protected Attributes> - This component indicates the characteristics that the system shall not consider in the decision-making process to ensure equitable treatment. A few examples of such characteristics are *race*, *gender* or *age*. The template guarantees that the system does not engage in direct discrimination by clearly stating these attributes. This component is optional as some fairness requirements can be defined without having to explicitly mention protected attributes.

And Proxy(IES)<Proxy(ies)> - This component represents variables that are part of anti-protected attributes but may indirectly represent protected attributes, such as *zip code* that correlate with race. Identifying proxies is crucial for mitigating indirect discrimination, as decisions made based on these proxies could unintentionally lead to biased outcomes. This component is optional since not every fairness requirement will involve proxies.

7.2 Meta-Model for Fairness Template

The meta-model for the fairness template as shown in Figure 15, offers a detailed, structured representation of how various components within a fairness framework interact. It was inspired by the MASTER meta-model published in the study titled *"Benchmarking Requirement Template Systems: Comparing Appropriateness, Usability, and Expressiveness [31]."* The original model provided a foundation that was adapted and extended to focus on fairness-related elements in software systems. While the original meta-model encompassed general requirements engineering concepts, this new meta-model was designed to represent the fairness template, explicitly outlining how different fairness components interact within software systems.

The meta-model provides an operational blueprint that outlines how fairness should be embedded into the system's functionality by translating the high-level requirements from the fairness template into detailed components and interactions within the software system. The meta-model is composed of both reused elements from the existing MASTER meta-model and newly defined components specifically tailored for fairness. The grey boxes denote reused components from the MASTER meta-model, while the white boxes represent newly introduced elements aimed at addressing fairness concerns within software systems.

One of the critical aspects of this meta-model is its ability to formalize fairness activities as part of the overall system processes. The model operationalizes fairness by introducing fairness-related activities such as detecting bias, monitoring fairness, and mitigating bias. These activities are not only tied to the system's primary functions but are also triggered and controlled by conditions and specific logical expressions defined within the model. This allows the fairness operations to be context-sensitive and adaptable based on the system's environment or specific events. By breaking down fairness into these manageable components, the meta-model ensures that fairness is embedded into the system architecture in a structured and measurable way. This guarantees that fairness is not just an afterthought but a critical element of the system's functionality.

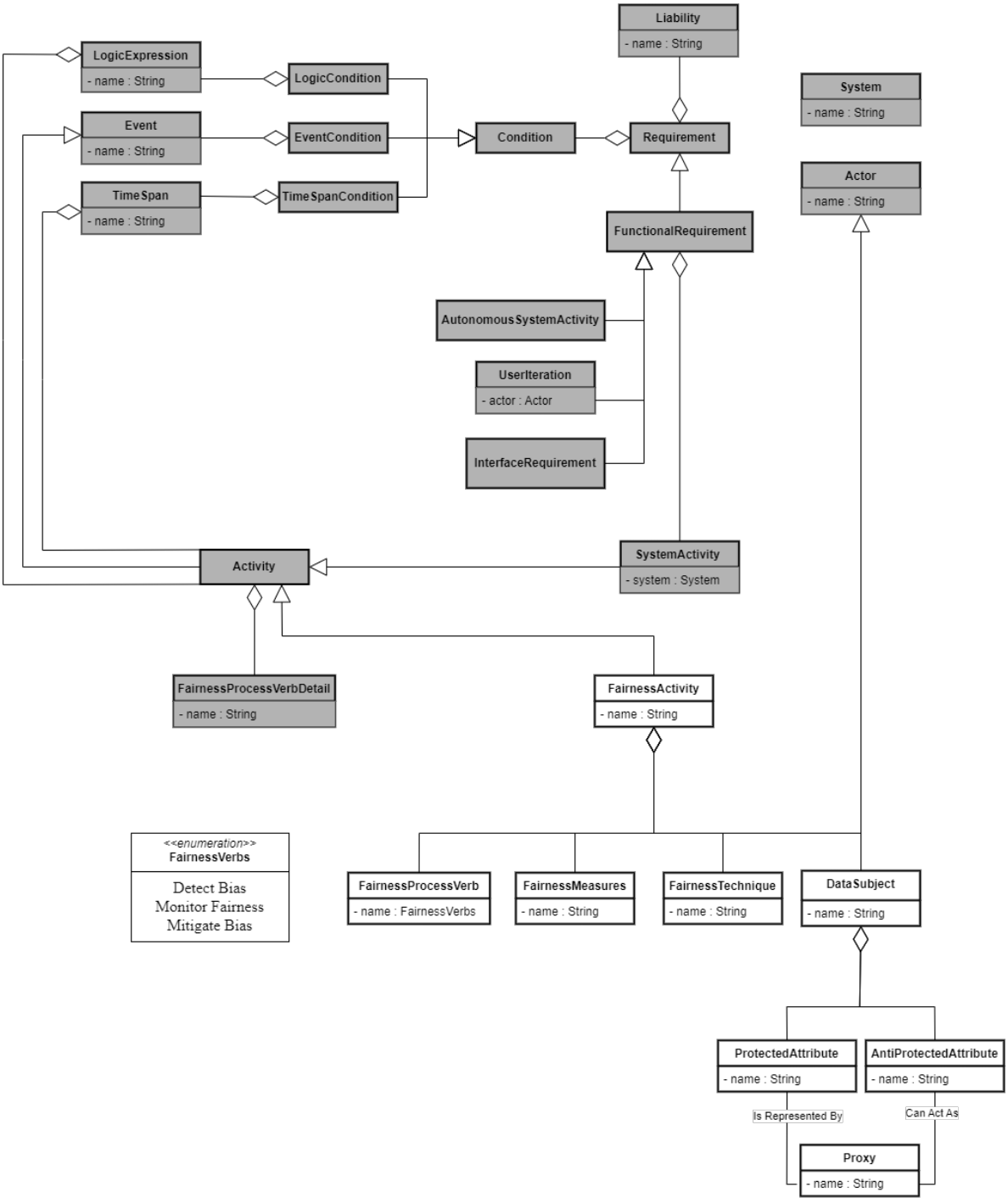

Figure 15: Meta-Model of the Fairness Template

Chapter 8

Test Case and Evaluation

This section aims to evaluate the feasibility of the proposed fairness template in real-world scenarios. It discusses the practical application of the template through specific test cases, such as healthcare insurance and banking systems, to showcase how fairness requirements can be systematically defined and implemented.

8.1 Healthcare Insurance System

System overview - The healthcare insurance system provides a digital platform for individuals to engage with a range of healthcare-related services. The system enables users to apply for health insurance, book appointments to seek medical consultations, submit claims, and examine insurance coverages. It is an automated decision system that uses algorithms to evaluate the applicant's eligibility, schedule appointments for medical services, approve or deny claims, and set insurance premiums.

Why Fairness is Relevant - Fairness is of paramount concern in healthcare insurance because automated decision systems choices, regarding healthcare accessibility, claims approval, and insurance premiums can be significantly impacted by protected attributes, such as race, gender, socioeconomic status, or medical history. For example, a biased algorithm could unjustly deny the claims of individuals belonging to a particular racial group or offer greater premiums to individuals of a specific gender group based on biased historical data. In order to prevent prejudice and guarantee that everyone has equitable access to healthcare, it is crucial that these decisions are made fairly.

Fairness measures in this system ensures that

- Decisions regarding insurance coverage are made impartially.
- Equal access to healthcare professionals is ensured through equitable scheduling of appointments.
- Claims are handled fairly, ensuring that all individuals obtain the financial assistance they need for medical expenses.

8.1.1 System Interactions

- **Apply for Healthcare Insurance** - Users submit personal information such as age, gender, medical history and income to request health insurance coverage. The information is utilized by the system to evaluate eligibility, determine coverage options, and establish premiums.
- **Schedule Appointments** - After obtaining the insurance, individuals have the ability to book appointments with healthcare providers. The system guarantees equitable access to available time slots, considering the level of urgency and individual health conditions.
- **Submit Claims** - Users have the ability to submit claims for reimbursement after they have received healthcare services. The system assesses these claims by considering factors like the service type, user's coverage, and medical necessity.

8.1.2 Requirements Defined Using Fairness Template for Healthcare Insurance System

Insurance Approvals

Requirement 1: <The healthcare insurance system> SHALL APPLY <disparate impact analysis> TO DETECT BIAS [IF the insurance approval rate disparity between racial groups exceeds 5%], ENSURING <demographic parity> AMONG <insurance applicants>, WHILE CONSIDERING <medical history>, WITHOUT DISCRIMINATING AGAINST <race> AND PROXY <geographic region>

Requirement 2: [EVERY 30 DAYS], <The healthcare insurance system> SHOULD APPLY <statistical parity> TO MONITOR FAIRNESS, ENSURING <equal opportunity> AMONG <insurance applicants>, WITHOUT DISCRIMINATING AGAINST <gender>

Requirement 3: <The healthcare insurance system> WILL APPLY <reweighting techniques> TO MITIGATE BIAS, ENSURING <disparate impact ratio> AMONG <insurance applicants>, WHILE CONSIDERING <income level>, AND PROXY <employment status>

Appointment Scheduling

Requirement 1: <The healthcare insurance system> SHALL APPLY <adversarial debiasing techniques> TO MITIGATE BIAS, ENSURING <treatment equality> AMONG <patients scheduling appointments>

Requirement 2: <The healthcare insurance system> WILL APPLY <subgroup fairness audits> TO MONITOR FAIRNESS [EVERY 60 DAYS], ENSURING <subgroup fairness>, AMONG <patients booking appointments>, WHILE CONSIDERING <medical urgency>, WITHOUT DISCRIMINATING AGAINST <geographic location>

Requirement 3: [IF the wait time disparity between different races exceeds 10%], <The healthcare insurance system> SHOULD APPLY <logistic regression with fairness constraints> TO DETECT BIAS, ENSURING <equalized odds> AMONG <patients scheduling appointments>

Claims Processing

Requirement 1: <The healthcare insurance system> SHALL APPLY <reweighting techniques> TO MITIGATE BIAS, ENSURING <demographic parity>, AMONG <insurance claimants>, WITHOUT DISCRIMINATING AGAINST <income level>

Requirement 2: <The healthcare insurance system> SHOULD APPLY <fairness through awareness audits> TO MONITOR FAIRNESS [EVERY 45 DAYS], ENSURING <equal opportunity> AMONG <insurance claimants>

8.2 Banking System: Loan Approval System

System overview - The loan approval system is intended for use by banks that provide its clients with credit lines, mortgages and personal loans. The system evaluates eligibility, interest rates, and approval decisions by processing financial and personal data submitted by users requesting loans. It is an automated decision system that uses algorithms to evaluate credit risk, establish loan terms and determine whether to approve or reject loan applications.

Why Fairness is Relevant - Ensuring fairness is important in loan approval systems because biased algorithms can unfairly impact specific demographic groups in loan approvals and interest rates. Bias against people due to their race, gender, income level can lead to unfair access to financial resources. Financial inequality may persist in the system if specific racial or socioeconomic groups are unfairly denied loans or assigned higher interest rates.

Fairness measures in this system ensures that

- The approval rates of loans remain constant for various demographic groups.
- Interest rates are assigned equitably based on creditworthiness, free from bias related to race or gender.
- Socioeconomic status and geographic location do not affect access to credit.

8.2.1 System Interactions

- **Apply for Loan** - Users submit personal information such as income, credit score, employment history to apply for loans. Using this information, the system evaluates the user's eligibility for the loan, determines their qualifying interest rate, and decides on loan approval or denial based.
- **Check Loan Status** - After submitting their application, users can monitor the status of their loan. The system conveys the decision (whether it's approved or denied) and gives additional information regarding the loan amount, interest rates, and repayment terms.

8.2.2 Requirements Defined Using Fairness Template for a Loan Approval System

Loan Approvals

Requirement 1: <The loan approval system> WILL APPLY <fairness constrained optimization>, TO DETECT BIAS [IF the loan approval rate disparity between racial groups exceeds 7%], ENSURING <demographic parity> AMONG <loan applicants>, WHILE CONSIDERING <credit score>

Requirement 2: <The loan approval system> SHALL APPLY <adversarial debiasing> TO MITIGATE BIAS, ENSURING <equalized odds> AMONG <loan applicants>

Loan Interest Rates

Requirement 1: <The loan approval system> SHALL APPLY <reweighing techniques> TO MITIGATE BIAS, ENSURING <equal opportunity> AMONG <loan applicants>, WITHOUT DISCRIMINATING AGAINST <geographic location>, WHILE CONSIDERING <debt-to-income ratio>

Requirement 2: [EVERY 60 DAYS], <The loan approval system> SHOULD APPLY <threshold adjustment > TO MONITOR FAIRNESS, ENSURING <Calibration Score> AMONG <loan applicants>, WHILE CONSIDERING <financial history>

Requirement 3: [IF the interest rate disparity between genders exceeds 5%], <The loan approval system> WILL APPLY <reject option classification> TO DETECT BIAS, ENSURING <predictive parity> AMONG <loan applicants>, WITHOUT DISCRIMINATING AGAINST <gender>

Chapter 9

Related Work

This chapter provides an overview of the existing literature and state of the art in fairness within software systems. The first section focuses on relevant papers that help in defining fairness in software systems, identifying challenges in ensuring fairness, and exploring techniques for bias detection and mitigation. The second section provides an overview of the current state of fairness research, showcasing the significant progress and ongoing challenges in the field.

9.1 Summary of Existing Literature

In this section, the existing literature has been categorized into five key areas to provide a structured understanding of fairness in software systems.

Fairness in Software Engineering and Requirements

Authors Yuriy Brun and Alexandra Meliou [12] argue that software fairness is a critical aspect of software quality and that it requires addressing various challenges in the software engineering process. The authors claim that software fairness is analogous to software quality and that it encompasses issues related to requirements, specification, design, testing, and verification. The paper also highlights the need for language, tool, and automation support at the levels of requirements, design, implementation, testing, and verification to ensure software fairness. *This is consistent with my thesis, which aims to establish fairness as a necessity by introducing a Fairness Template. While Brun and Meliou promote fairness throughout the software development lifecycle, this research concentrates on putting fairness into practice during the requirements engineering stage, bridging a gap in their overall framework by presenting a template for specifying fairness related requirements.*

Luciano Baresi, Chiara Criscuolo, and Carlo Ghezzi [5] argue that fairness is a non-functional software property that should be treated as a first-class entity throughout the entire software engineering process. They emphasize the importance of fairness in the context of ML software, which is increasingly used to make decisions that affect people's lives. The paper offers a comprehensive survey of existing studies on fairness testing of ML software, focusing on the testing workflow and testing components related to fairness testing. The authors also define fairness bugs and fairness testing, which aim to uncover fairness bugs through code execution and guide software repair efforts to address fairness issues. *Unlike their research that focuses on testing and debugging, this thesis stresses fairness requirements.*

Challenges in Defining and Ensuring Fairness

Agarwal et al. [1] propose a novel *fairness score* and *bias index* aimed at detecting and mitigating biases in AI systems. The authors argue that current reliance on statistical measures is insufficient and calls for process standardization to comprehensively address fairness and accountability. This work emphasizes the need for a structured approach to measure and mitigate bias in AI applications. *The author's emphasis on standardization highlights the need for structured templates, like the one proposed in this thesis, to address fairness from the early stages of software development.*

Sahil Verma and Julia Rubin [77] discuss the complexities and ambiguities surrounding fairness definitions in machine learning and automated decision-making systems. The authors argue that fairness definitions can be subjective and open to interpretation, leading to scenarios where the same case can be considered fair according to some definitions and unfair according to others. They provide an intuitive analysis of various fairness definitions, such as individual fairness, group fairness, statistical parity, equal opportunity, and equalized odds, highlighting the challenges in ensuring fairness in practice. *This thesis takes this into consideration by providing flexibility in the fairness template, enabling the user to specify different fairness goals, depending on the context.*

Jørgensen and Søgaard [40] critique the Rawlsian approach to fairness in AI, identifying several loopholes that allow systemic biases to persist. They suggest that these loopholes can be exploited, thereby undermining the fairness principles. The authors recommend revisions to the Rawlsian framework to better tackle fairness in AI systems. *While the authors in this paper use a more philosophical approach to fairness, this thesis concentrates on implementing fairness using specific requirements engineering techniques. The fairness template created in this thesis is based on concrete fairness definitions, instead of theoretical philosophical ideas.*

Friedler et al. [25] propose a comprehensive theoretical framework for understanding fairness in machine learning. They categorize fairness definitions into group and individual fairness, discussing their implications and trade-offs. *This theoretical foundation was crucial as it helps in evaluating different fairness approaches thereby ensuring its adaptability across multiple scenarios.*

Bias Detection and Mitigation Techniques

Authors Simon Caton and Christian Haas [14] provide an overview of different approaches to mitigate biases and enhance fairness in machine learning. The authors categorize these approaches into pre-processing, in-processing, and post-processing methods, further subdividing them into 11 method areas. They discuss fairness considerations not only in binary classification but also in regression, recommender systems, and unsupervised learning, offering insights into various open-source libraries available. This paper concludes by outlining five dilemmas for fairness research, providing a comprehensive understanding of the complexities surrounding fairness in machine learning. *While they focus on mitigating bias during and after development, my thesis tackles fairness before development by incorporating fairness directly into the requirements using the Fairness Template.*

Zhang, Lemoine, and Mitchell [83] propose the use of adversarial learning techniques to mitigate unwanted biases in AI systems. They develop a framework that integrates adversarial debiasing into the model training process, demonstrating its effectiveness in reducing bias without significantly compromising performance. This work provides practical solutions for addressing biases in AI applications. *The fairness template doesn't address adversarial learning or other post-processing techniques.*

Bellamy et al. [7] present the AI Fairness 360 toolkit, an extensible framework designed to help developers detect, understand, and mitigate unwanted algorithmic bias in AI systems. The toolkit includes a comprehensive set of metrics and bias mitigation algorithms, making it a valuable resource for practitioners working on fairness in AI. This work emphasizes the importance of practical tools in achieving fairness in AI development. *The tool proposed in this paper is a post-development tool, whereas this thesis focuses on addressing fairness earlier in the SDLC.*

Mehrabi et al. [57] provide an extensive survey on bias and fairness in machine learning, covering various types of biases, their sources, and the impact on AI systems. They discuss existing fairness definitions and metrics, as well as techniques to mitigate bias. This comprehensive review highlights the complexities of achieving fairness in machine learning and suggests areas for future research. *This work contributes to the development of the Fairness Template in this thesis, which is intended to support various fairness metrics based on the system context.*

Ethical Considerations and Regulatory Challenges

Crawford and Calo [19] explore the ethical implications of fairness in AI systems. They argue that technical solutions alone are insufficient and must be complemented by ethical considerations and regulatory frameworks. Their work underscores the importance of a multidisciplinary approach to addressing fairness in AI. *The primary focus of the thesis to offer a technical solution like the fairness template, but it recognizes the significance of ethical considerations by incorporating few components to protect the data subject, ensuring the system aligns with broader societal and ethical values.*

Richardson, Schultz, and Crawford [69] investigate the impact of predictive policing algorithms on fairness. They highlight how these systems can perpetuate existing biases and disproportionately affect marginalized communities. The authors call for greater transparency and accountability in the deployment of predictive policing technologies. *By Incorporating fairness audits in the fairness conceptual model, it allows for the continuous monitoring and verification of fairness, addressing concerns raised by the authors regarding the opaque nature of software systems.*

Binns [9] explores the concept of fairness in machine learning through the lens of political philosophy. The paper discusses different fairness definitions derived from philosophical theories and evaluates their applicability to AI systems. Binns argues that a deeper understanding of these philosophical foundations can inform the development of fairer AI systems. *Binns' philosophical ideas contribute to shaping the creation of the fairness template in this thesis, enabling a flexible approach to defining and implementing fairness. However, the emphasis is on practical application rather than delving deeply into the philosophical foundations.*

Cossette-Lefebvre and Maclure [17] focus on the issue of wrongful discrimination in AI, discussing the inherent biases and the necessity for explainability to uphold fairness principles. They highlight the risks associated with unaddressed biases and stress the importance of transparent AI systems to mitigate wrongful discrimination. *This thesis directly addresses this concern by incorporating components like protected attributes, proxies and risk related components. These components consider all possible sources that pose a threat to the system, ensuring they are accounted for and are not used to generate unfair decisions.*

Trade-offs in Fairness

Suresh and Guttag [75] offer a framework to understand the unintended consequences of machine learning, including biases that may arise at different stages of the machine learning pipeline. They discuss the historical context of these biases, and trade-offs in fairness definitions and propose strategies to mitigate them, emphasizing the need for a holistic approach

to fairness in AI. *These trade-offs are acknowledged in this thesis where the fairness template allows the stakeholders to specify multiple fairness measures and prioritize them based on the system's objective.*

Corbett-Davies and Goel [16] analyze the trade-offs between fairness constraints and the performance of AI models. They argue that imposing fairness constraints can sometimes lead to reduced accuracy, but these trade-offs are necessary to achieve equitable outcomes. This work provides a nuanced view of the challenges in balancing fairness and performance in AI systems. *Their work adds to the fairness template created in this thesis by allowing engineers to specify fairness constraints and document potential performance impacts, which ensures that that the trade-offs are transparent and managed appropriately.*

9.2 State of the Art

This section explores the current state of the art in ensuring fairness in software systems, focusing on the four key stages of fairness-related practices. These stages are critical components in the development of fair software systems and span different phases of the software development lifecycle, from requirements engineering to system testing and verification.

9.2.1 Detecting Discrimination

As the reliance on software systems for critical decisions continues to grow, the importance of identifying and addressing biases embedded in these systems has become increasingly apparent. Discrimination can emerge in various forms, often as a result of biased data, flawed algorithms, or systemic issues that inadvertently favor certain demographic groups over others [42]. The process of detecting discrimination involves a series of methodologies and tools designed to uncover these biases early in the software development lifecycle, allowing developers to address them before they become deeply ingrained in the system.

Methods and Tools for Detecting Discrimination

To address the challenge of detecting discrimination, several methods and tools have been developed that focus on analyzing both data and algorithms for biases related to protected attributes such as race, gender, age, and socioeconomic status. These methods offer a systematic approach to pinpoint possible sources of bias, empowering developers to make necessary corrections prior to the software being deployed.

Fairness-Aware Data Mining - This approach includes examining datasets to identify patterns of biases that may result in discriminatory outcomes [44]. For instance, if a dataset used to train a hiring algorithm overrepresents a particular gender or racial group, the resulting model may reflect these biases in its predictions. Fairness-aware data mining techniques scan datasets for such disparities, helping to ensure that the training data is balanced and representative of all relevant groups [44].

IBM's AI Fairness 360 (AIF360) toolkit is a well-known tool that employs fairness-aware data mining [7]. This open-source library aims to assist developers in detecting and mitigating bias in machine learning models. AIF360 offers a range of fairness metrics and algorithms for developers to evaluate how various demographic groups are impacted by a model. AIF360 offers

developers a wide range of tools to assess fairness, enabling them to detect biases at an early stage of development and resolve them before deploying the software.

Disparate Impact Analysis - This method is used to identify whether a specific algorithm disproportionately affects individuals belonging to a protected group [42]. In software development, disparate impact analysis entails evaluating how an algorithm affects various demographic groups to uncover any notable differences in outcomes.

Google's What-If Tool is an example of a tool that facilitates disparate impact analysis. The What-If Tool helps users visualize how machine learning models perform with various demographic groups, enabling them to identify if some groups are disproportionately affected by the model's predictions [81].

Counterfactual Fairness Testing - This method involves assessing whether an individual's outcome would change if they belonged to a different demographic group, assuming all other factors remain constant [50]. This method is especially helpful in detecting underlying biases that may not be obvious using standard fairness measures.

A study by Kusner et al. (2017) introduced the concept of counterfactual fairness, applying it to a recidivism prediction algorithm used in the criminal justice system [50]. The study demonstrated that the algorithm could be considered fair if an individual's predicted likelihood of reoffending would remain the same if their race were different, provided all other relevant variables were held constant [50]. This counterfactual fairness testing method is successful in identifying biases that other methods may miss, providing a more thorough analysis to ensure fairness in decision-making.

Bias Detection in Textual Data - Textual content, especially within the context of natural language processing (NLP), can serve as a major source of bias. Identifying prejudice in written content requires examining language models for biased word connections and stereotypes [32]. For instance, word embeddings, mathematical representations of words used in NLP can inadvertently encode societal biases, such as associating male pronouns more strongly with career-related terms and female pronouns with family-related terms [11].

Tools such as the Word Embedding Association Test (WEAT) are employed to uncover biases in NLP models, aiding in the recognition and resolution of discrimination in language processing systems [76]. Developers can guarantee that their NLP systems do not propagate harmful stereotypes or biases by assessing the fairness of word associations in these models.

Real-World Examples of Detecting Discrimination

Facebook's Ad Delivery System - Facebook's ad delivery system unfairly targets ads based on attributes like race and gender, although advertisers have not specified these parameters [2]. Disparate impact analysis was employed to identify that certain demographic groups were being targeted with discriminatory housing and employment ads [2].

Twitter's Image Cropping Algorithm - It was found that Twitter's automatic image cropping algorithm favored people with lighter skin tones when cropping images, regardless of the presence of other prominent figures in the image [82].

Google's Image Recognition Software - Google's system exposed racial bias when it was found to inaccurately classify images of individuals belonging to specific ethnic communities [1]. Through fairness-aware data mining, researchers discovered biases in the training data that resulted in the underrepresentation of certain ethnic communities.

[1] https://algorithmwatch.org/en/google-vision-racism/

9.2.2 Mitigating Discrimination

Mitigating discrimination is an essential part of developing fair and equitable software systems. After identifying the potential sources of bias in the detection phase, the next important task is implementing strategies and tools that actively reduce or eliminate these biases. Mitigation seeks to prevent software systems from perpetuating or exacerbating existing inequalities, particularly in sensitive areas like hiring, banking, and law enforcement [6]. This section explores different tools for mitigating discrimination and offers real-world examples.

Tools for Mitigating Discrimination

Software developers can now mitigate discrimination with various readily available tools. These tools provide developers with an efficient means to incorporate fairness into software systems. This section covers a few tools that help mitigate discrimination.

Fairlearn - Fairlearn is an open-source toolkit developed by Microsoft that provides tools for assessing and improving the fairness of machine learning models [10]. It consists of mitigation techniques like *Exponentiated Gradient Reduction*, which adjusts the model decision boundaries to find the optimal balance between accuracy and fairness. Fairlearn also provides developers with visualization tools that facilitate comprehending the effect of various fairness interventions on the model performance [80].

Themis-ML - The University of Chicago developed Themis-ML, which consists of a range of pre-processing, in-processing, and post-processing techniques to detect and mitigate bias [55]. Themis-ML works incredibly effectively when developers must balance two or more fairness requirements, such as ensuring both demographic parity and equalized odds [4]. Themis-ML provides a versatile framework to evaluate various mitigation strategies, assisting developers in building accurate and equitable models.

Google's TensorFlow Fairness Indicators - Google's TensorFlow Fairness Indicators is yet another tool that aids in detecting and mitigating discrimination in machine learning models. This tool integrates seamlessly with TensorFlow, a well-known machine learning framework, and offers a variety of fairness metrics to evaluate a model's performance across various demographic groups[2]. TensorFlow Fairness Indicators work best in production scenarios where model fairness needs to be continuously monitored. This tool can handle complex scenarios involving multiple protected characteristics and process large-scale, distributed data. TensorFlow Fairness Indicators provide developers with precise details on the model's performance, enabling them to implement targeted mitigation strategies to address specific fairness problems.

Real-World Examples of Mitigating Discrimination

Several prominent examples demonstrate how software systems mitigate discrimination using the above mentioned approaches and tools.

LinkedIn's Recruiter Tool - LinkedIn's recruiter tool, designed for job matching and candidate searches, initially demonstrated a prejudice that preferred male candidates over female candidates for specific roles [73]. To address this issue, developers incorporated some in-processing techniques to reduce gender bias by altering the model's learning process. Post-processing

[2] `https://www.tensorflow.org/tfx/guide/fairness_indicators`

techniques were also used to re-rank the applicants and improve the representation of women in search results [27].

The UK's Home Office Visa Application Algorithm - UK's Home Office Visa Application Algorithm was discovered to discriminate against applicants from specific countries [41]. The Home Office conducted a thorough evaluation of the system, following which they employed pre-processing techniques on the training data to devoid it of any geographical bias. Additionally, they used post-processing techniques to fine-tune the algorithm's decision-making processes [51].

Airbnb's Anti-Discrimination Efforts - Airbnb exhibited racial bias when guests from specific racial backgrounds were unfairly denied bookings [23]. As a response, Airbnb partnered with several developers and implemented numerous mitigation strategies to address these prejudices. One such strategy was *Project Lighthouse*, which sought to identify and quantify racial prejudice on the platform[3]. Also, Airbnb penalized hosts who often discriminated against guests and tweaked the system to encourage the fair treatment of all users [59].

9.2.3 Monitoring Discrimination

Monitoring discrimination is essential to the software development lifecycle, particularly when implementing machine learning models or automated decision-making systems. After a software system is deployed, it moves into a complex and uncertain environment where it interacts with data that differs from the training data [15]. Moreover, biases may be introduced or made worse by user interactions, feedback loops, and system changes [65]. Therefore, to ensure that the system remains equitable and does not inadvertently cause harm to any group in the long run, discrimination must be continuously monitored. Monitoring includes tracking the system's performance across different demographic groups, auditing outcomes on a regular basis, and setting up feedback loops to identify and address any biases that may arise [65]. This process maintains the system's integrity and fosters trust among users and stakeholders. This sub-section covers the tools required to monitor discrimination and real-world examples of continuous fairness monitoring being incorporated into the software system.

Methods and Tools for Monitoring Discrimination

The methods for monitoring discrimination can broadly be classified into when and how they are applied, the type of data they analyze, and the specific fairness metrics they use. Several tools have been designed and developed to assist the developers in monitoring discrimination in their software systems. These tools offer a range of functionalities, from real-time auditing to periodic reporting, and are designed to integrate seamlessly into the operational environment.

Real-Time Monitoring - Real-time monitoring pertains to continuous monitoring of software systems, particularly machine learning models that run in production environments. This approach is essential for identifying biases early on and correcting them, particularly in dynamic environments where the input data may face significant change over time [22]. Organizations can immediately identify deviations from intended behavior when an algorithm disproportionately favors or disfavors particular demographic groups by monitoring important fairness metrics in real-time [42]. Real-time monitoring commonly involves using dashboards that display the system's performance based on various fairness metrics and automated alerts that inform developers or operators when certain thresholds are exceeded.

[3] `https://www.nytimes.com/2020/09/24/travel/airbnb-pandemic.html`

Fiddler AI is a real-time monitoring platform, particularly useful in sectors like finance and healthcare, where real-time decisions have a significant impact and fairness is a critical concern [46]. When operating in production, the tool helps identify bias and drifts in the model by tracking critical performance and fairness metrics across various demographic groups. It also quickly triggers alerts if it detects any potential bias. With its advanced features, it can pinpoint the underlying source of bias in real-time, making it identify the root cause of a bias.

Periodic Audits - Periodic Audits entail having regular checks at predetermined times, such as weekly, monthly, or quarterly, to evaluate the system's fairness [72]. Periodic audits are better suited for systems that do not require continuous monitoring but still need to be reviewed on a regular basis to ensure they adhere to fairness criteria. Organizations may do a batch evaluation of data gathered over a time interval as part of these audits, examining how the system's output affects various demographic groups [6]. They may also perform Regression testing for fairness to ensure that new modifications do not generate unintentional biases while updating the system [49].

Audit-AI is a tool developed to conduct periodic audits of machine-learning models to ensure fairness. It is best suited for sectors that rely on human resources, where periodic evaluations are critical to ensure the system remains fair and unbiased. It streamlines the process of detecting biases by examining the outcomes of models among various demographic groups, and it also monitors the changes in fairness over time. This tool offers an in-depth analysis of the identified biases, highlights areas that may contain potential bias, and suggests ways to address them[4].

Feedback Loops and Post-Deployment Testing - Feedback loops and post-deployment testing require gathering and evaluating data from users or external inputs following the implementation of the system. This approach assists companies in capturing real-life experiences and identifying biases that may not have been apparent during the early stages of testing [28]. Using feedback loops, decision-making biases in the system are found and rectified by collecting user feedback. Contrarily, post-deployment testing entails putting the system through fictitious or edge-case situations to see if it yields equitable results in novel circumstances [37].

H2O.ai's Driverless AI offers continuous monitoring and real-time modifications based on customer feedback and updated data inputs[5]. It is a crucial tool for companies that value continuous fairness and transparency in their AI models since it offers tools for conducting post-deployment tests to guarantee the system's fairness when faced with new data or operating conditions.

Real-World Examples of Monitoring Discrimination

Several organizations have already put monitoring systems in place to guarantee that their models stay fair as time passes. Below are real-world instances demonstrating companies' ongoing efforts to monitor discrimination within their AI systems.

HaterNet - HaterNet is a real-time monitoring system created to identify and address online hate speech and discriminatory content, currently used in the Spanish National Office against hate crimes of the Spanish State Secretariat [61]. It was created in response to the growing issue of hate speech on social media platforms. It uses machine learning algorithms to monitor user-generated content for offensive language, harmful behaviors, and discriminating words.

[4] https://run.unl.pt/handle/10362/138796

[5] https://docs.h2o.ai/driverless-ai/1-8-lts/docs/booklets/DriverlessAIBooklet.pdf

HackerNet scans data and flags any content that exhibits discriminatory behavior towards certain people or groups based on race, gender, religion, or other protected characteristics [61].

Uber's Driver-Partner Matching System - Uber regularly monitors its driver-partner matching algorithms to make sure that drivers and customers are treated fairly. Preventing pricing and availability-based discrimination based on gender, ethnicity, or geography is a significant challenge [26]. Uber has a monitoring system to oversee fairness, tracking how the matching algorithm allocates surge pricing and ride requests across various demographic groups [26]. By conducting routine checks and monitoring in real-time, Uber addresses the concerns of the marginalized community, who may face longer wait times or surge prices due to biased algorithms.

YouTube's Recommendation System - YouTube continuously monitors the fairness of its recommendation system to rectify any potential biases and guarantee suggestions for diverse and equitable content consumption. YouTube's recommendation system employs TensorFlow Indicators to track and audit the system to prevent any preferential treatment based on race, gender, or other protected attributes [18].

9.2.4 Explaining Algorithmic Decisions

Transparency, accountability, and trust in AI systems are all dependent on the ability to provide explanations for algorithmic decisions, especially where machine learning models and algorithms make significant decisions. Algorithmic decision-making often suffers from a lack of clarity, giving rise to the issue of the *black box problem*, where the functioning of the model remains obscure. The lack of transparency can result in mistrust, especially by individuals who are being treated unjustly by the system. Explainability entails providing understandable explanation of how a specific decision was reached by an algorithm, this enables the stakeholders to evaluate the fairness, reliability and accuracy of the system. Fair and equitable applications result from transparent systems that promote better trust and facilitate the detection of possible biases in the decision-making process.

Tools for Explaining Algorithmic Decisions

Several tools have been created to aid the understanding of the reasoning behind decisions made by machine learning models. These tools offer various techniques to convert complex, multi-dimensional models, such as deep neural networks into results that are comprehensible to humans.

LIME (Local Interpretable Model-Agnostic Explanations) - LIME is an agnostic tool that is widely used to help clarify predictions made by machine learning models by locally approximating it with an interpretable model. LIME operates by modifying the input data surrounding the prediction and monitoring the changes in predictions [68]. It proceeds to fit a more basic, easily understood model, such as linear regression, to the modified data to clarify the decision based on the most important features [68].

LIME has been used in healthcare to explain the predictions of a neural network tasked with diagnosing pneumonia from chest X-rays [38]. LIME helped doctors build trust in the system's diagnostic capabilities by showing which areas of the X-ray were considered important for the model's decision, allowing them to understand and verify the model's output [38].

SHAP (SHapley Additive exPlanations) - SHAP is a popular tool used for clarifying decisions made by machine learning models, which is built upon the idea of Shapley values from cooperative game theory. SHAP values evaluate the significance of specific features by assessing their impact on the model's prediction [52]. In order to get the best results, every potential feature combination is taken into account, and its effect on the model's output is computed [52]. This offers an interpretability framework at both the global and local levels, allowing SHAP to clarify individual predictions and offer insight into how features impact the model.

Financial institutions utilize SHAP to provide explanations for credit scoring decisions. For instance, SHAP was used to determine which characteristics, such as income, debt, or credit history were most important in the decision to deny a customer a loan, thereby explaining the denial [58].

IBM Watson OpenScale - IBM Watson OpenScale offers an end-to-end AI management platform that tracks model drift, fairness, and accuracy in real time and provides thorough explanation for each prediction[6]. The system includes a dashboard that is easy for users to navigate, showing important features impacting decisions and offering explanations accessible to those without technical expertise.

IBM Watson OpenScale is used in healthcare where transparency is critical. In one instance, Watson OpenScale was employed to explain AI-generated treatment plans for cancer patients by showcasing the critical clinical data, such as lab results, genetic markers, and medical history that influenced the model's suggestions [60]. These explanations assist healthcare providers in comprehending and having confidence in the decisions made by AI.

Real World Examples of Explaining Algorithmic Decisions

Google's AdSense - Explainability helps the Google AdSense platform make more transparent decisions by generating more targeted ads based on user profiles and website content. By using the SHAP tool, advertisers can now visualize the factors that impacted the ad-serving process, such as user behavior, demographics, site content. This transparency has enabled advertisers to boost their Return on Investment and ad targeting.

JPMorgan Chase's Credit Decision Models - JPMorgan Chase employs automated decision systems to evaluate the creditworthiness of prospective borrowers. Owing to the delicate nature of credit choices, the bank implemented explainability strategies like Jade (JPMorgan Chase Advanced Data Ecosystem) and Infinite AI to offer explanations for credit decisions made for specific applicants [7]. This decision helped JPMorgan Chase enhance its transparency and trust amongst customers and regulators.

Tesla's Autopilot System - Since machine learning models are a major component of Tesla's Autopilot system, they have employed explainability tools to help developers comprehend the system's actions in real-time, particularly during high-stakes situations like crashes or near-misses. Tesla uses a proprietary tool to examine sensor data, vehicle behavior, and real-time environmental inputs to provide explanations for the actions taken by the Autopilot system.

[6] https://mediacenter.ibm.com/media/Manage+AI+models+with+IBM+Watson+OpenScale/1_lqiuc2uq

[7] JPMorgan Chase: Digital transformation, AI and data strategy sets up generative AI

Chapter 10

Conclusion and Future Work

In conclusion, this thesis addresses the critical challenge of integrating fairness considerations into software systems from the early stages of development, particularly during the requirements engineering phase, by offering a structured method that emphasizes the importance of considering fairness throughout the entire software development process rather than treating it as an afterthought.

By conducting a systematic literature review, the essential core ideas for specifying fairness-related requirements were identified and organized, which addressed *RQ1*. This led to the creation of an extensive glossary that laid the groundwork for the subsequent development of the fairness template.

RQ2 focused on how the identified fairness concepts could be represented in a conceptual model. The fairness conceptual model developed in this thesis drew inspiration from an established security model but was specifically tailored to address fairness risks in software systems. This conceptual model became crucial in understanding the interaction of fairness concepts and guided the creation of the fairness template.

RQ3, which examined how the MASTER template could be modified to incorporate fairness concepts, was answered by adapting the Functional MASTER Template. The developed fairness template enables inexperienced requirements engineers to define fairness-related requirements, making fairness an integral part of the software requirements process.

Lastly, *RQ4* was addressed by applying the fairness template to real-world test cases, such as healthcare insurance and banking systems. These examples showed how the fairness template can be applied in real-world scenarios to define fairness requirements in different systems.

Limitations

Despite the significant strides made in this thesis, there are certain limitations that must be acknowledged. These limitations are directly tied to the scope and constraints of the thesis and offer insight into potential areas of improvement.

The main limitation of this thesis is that the defined requirements were evaluated for feasibility but not for usability. While the feasibility of creating fairness requirements was tested, the absence of domain experts or stakeholders means there was no external validation of the requirements' practicality.

Another limitation is that this thesis focused solely on the Functional MASTER Template. While this template was instrumental in guiding the development of fairness requirements, it does not address fairness requirements related to performance, reliability, and other non-functional aspects of software systems.

Finally, due to the limited duration of this thesis, several aspects of fairness in software systems could not be fully explored. For instance, real-time fairness monitoring was not addressed in the fairness template, and the incorporation of ongoing fairness tracking methods remains an open area for future research.

Future Work

The limitations highlighted above provide opportunities for further exploration, and by addressing these gaps, the work on fairness requirements in software systems can be significantly enhanced.

To validate the fairness templates practical application, stakeholders and domain experts need to assess the usability of this template in real-world scenarios. This would help determine how well the template performs in various scenarios and offer opportunities for additional customization.

The fairness template's scope could be expanded in the future to incorporate non-functional requirements. This would make it possible to incorporate fairness considerations into contexts pertaining to performance, reliability, and security in addition to functional features, ensuring that fairness is present throughout the software development lifecycle.

Lastly, fairness is an evolving concept, and future work must keep pace with the new risks and challenges it may pose as software systems continue to develop. One aspect that was not fully addressed in this thesis is real-time fairness monitoring. As systems engage with constantly evolving data, the ability to identify and mitigate biases in real time will become increasingly critical. Future work may explore expanding the fairness template to include functions for automated fairness validations, ensuring that evolving systems can adapt and maintain fairness throughout their lifecycle.

Appendix

Appendix A: Glossary

Terms	Definition	Alternate Terms/Alias	Source	Example
Fairness Goals	Principles and objectives that are established to ensure that actions and decisions are impartial and equitable across different contexts	Fairness Objectives	Fairness and Machine Learning: Limitations and Opportunities (2023)	Implementing a loan approval process that ensures impartiality by not discriminating based on protected characteristics
Fairness Requirement	Specific, actionable requirements derived from fairness goals, which outline what needs to be implemented in the system to achieve the fairness goals	-	On Adaptive Fairness in Software Systems (2021)	A credit scoring system must only consider financial history and exclude protected characteristics when approving loans
Fairness Technique	Methods or algorithms implemented to ensure that the fairness requirements are met	Fairness Control	Biases and Fairness in Deep Learning Models: A Survey on Inculcating Fairness in Generative Models (2023)	Implementing adversarial debiasing to detect bias in the loan approval system
Fairness Constraints	Specific limitations or rules imposed on a system or decision-making process	Ethical Constraints	Algorithmic indirect discrimination, fairness and harm 2022)	The loan approval rates for minority applicants must be within a 5% range of the approval rates for non-minority applicants

Terms	Definition	Alternate Terms/Alias	Source	Example
Fairness Audits	Regular, systematic evaluations of the system to check for compliance with fairness requirements	Bias Audits, Fairness Evaluation	AI's fairness problem: understanding wrongful discrimination in the context of automated decision-making (2022)	Conducting quarterly fairness audits to review loan approval outcomes and ensure no group is disadvantaged
Fairness Measures	Metric used to evaluate and ensure that a process or system is fair and unbiased	Fairness Metrics	A Survey on Bias and Fairness in Machine Learning (2021)	Disparity Index - Quantifies the difference in loan approval rates between different demographic groups
Group Fairness	Ensuring that decision-making processes are equitable across different predefined groups, typically defined by protected attributes	Statistical Fairness, Demographic Equity	In-Processing Modeling Techniques for Machine Learning Fairness: A Survey (2023)	The loan approval rate for men and women should be similar
Statistical Parity	Ensures that the decision rate is the same across groups defined by a protected attribute	Demographic Parity	In-Processing Modeling Techniques for Machine Learning Fairness: A Survey (2023)	Approving a loan should be made at the same rate for everyone, regardless of their group
Equal Opportunity	Ensures that the true positive rate is the same across different groups	-	In-Processing Modeling Techniques for Machine Learning Fairness: A Survey (2023)	Individuals who qualify for a loan should have an equal chance of receiving it, no matter their group
Equalized Odds	Ensures that both the chance of a correct decision and the chance of a mistake are the same for all groups	-	Fairness Definitions Explained (2018)	The loan approval system's performance (both in making correct and incorrect loan decisions) is not biased toward any specific demographic group.

Terms	Definition	Alternate Terms/Alias	Source	Example
Predictive Equality	Ensures that the false positive rate is the same across different groups	-	Fairness Definitions Explained (2018)	The likelihood of predicting someone to default on a loan is the same for all groups
Predictive Parity	Ensures that the positive predictive value is the same across different groups	Outcome Test	Fairness Definitions Explained (2018)	When the system predicts loan repayment, it's equally correct across all groups
Conditional Demographic Parity	Extends demographic parity by conditioning on a set of legitimate variables that are deemed acceptable to influence the decision	Conditional Statistical Parity	Fairness Definitions Explained (2018)	Ensures fairness by considering additional relevant factors like education level when making decisions
Treatment Equality	Measures the ratio of false negatives to false positives, demanding it is the same across groups	-	Fairness Definitions Explained (2018)	The ratio of overlooking qualified candidates to correct decisions is the same across groups
Calibration	Ensures that groups receive the correct probabilities of positive outcomes	-	On the Applicability of Machine Learning Fairness Notions (2021)	If a model predicts a 70% chance of success for a group of people, then approximately 70% of those people should indeed succeed
Individual Fairness	Focuses on treating similar individuals similarly	Similarity-Based Fairness	A Survey on Bias and Fairness in Machine Learning (2021)	If customer 1 is approved for a loan, customer 2 should also be approved, assuming their circumstances are comparable

Terms	Definition	Alternate Terms/Alias	Source	Example
Fairness Through Awareness	A fairness framework that ensures models are aware of the fairness constraints in relation to protected attributes at training time	-	Fairness Definitions Explained (2018)	Implementing a loan approval system that includes an awareness mechanism to adjust for socio-economic factors, ensuring that applicants from disadvantaged backgrounds are not unfairly disadvantaged
Fairness Under Unawareness	A fairness approach that involves ignoring protected attributes during the decision-making process to prevent bias	-	Fairness Definitions Explained (2018)	Loan approval algorithms are designed to be explicitly aware of and adjust for protected attributes to prevent discriminatory outcomes
Counterfactual Fairness	A fairness metric based on the idea that a decision is fair if it would be the same in the actual world and a counterfactual world where a sensitive attribute among the protected attributes is different	-	Contrastive Counterfactual Fairness in Algorithmic Decision-Making(2022)	Designing a loan approval model that evaluates whether an applicant would have received the same decision if their race, gender, or other sensitive attributes were different
Protected Attributes	Attributes that are legally protected from discrimination	Sensitive Attributes, Protected Characteristics, Protected Variables	Did You Do Your Homework? Raising Awareness on Software Fairness and Discrimination (2021)	Race or Gender, when approving loans
Anti-Protected Attributes	Attributes or variables that should not be protected under fairness considerations	Non-sensitive Attributes, Anti-Protected Characteristics	Did You Do Your Homework? Raising Awareness on Software Fairness and Discrimination (2021)	Income, debt levels, and credit history when approving loans

Terms	Definition	Alternate Terms/Alias	Source	Example
Proxy Discrimination	The inadvertent use of variables, while not explicitly sensitive, closely correlate with protected attributes and perpetuate bias	Indirect discrimination	Avoiding Discrimination through Causal Reasoning (2018)	Zip codes or educational history
Exploratory Variables	An attribute that also is highly associated with a protected attribute, and unequal treatment based on this attribute is considered acceptable and legitimate	-	Machine Learning Fairness Notions: Bridging the Gap with Real-World Applications (2022)	-
Data Subject	Individuals whose data is being processed and analyzed	-	Privileged and Unprivileged Groups: An Empirical Study on the Impact of the Age Attribute on Fairness (2022)	Customers in a bank
Stakeholder Involvement	Ensure that diverse perspectives are considered, making the decision-making processes understandable to all stakeholders	-	Can Fairness be Automated? Guidelines and Opportunities for Fairness-aware AutoML (2024)	Providing clear explanations for loan approval decisions, including the factors considered and the weight of each factor
Threat	Potential events or actions that exploit vulnerabilities to cause unfair outcomes	-	"An integrated conceptual model for information system security risk management supported by enterprise architecture management (2018)	A biased loan approval system that disproportionately rejects candidates from certain ethnic backgrounds

Terms	Definition	Alternate Terms/Alias	Source	Example
Vulnerabilities	Weaknesses in the system that can lead to unfair outcomes	Weakness, Flaws	An integrated conceptual model for information system security risk management supported by enterprise architecture management (2018)	Bias in training data, algorithmic bias, use of inappropriate proxies
Risk	The potential or actual adverse effects of unfair treatment	Harm	An integrated conceptual model for information system security risk management supported by enterprise architecture management (2018)	Economic loss, reduced opportunities. Denial of loans to qualified individuals
Impact	The outcomes that occur when detected risks become a reality in the system	Consequence	An integrated conceptual model for information system security risk management supported by enterprise architecture management (2018)	The bank may face financial loses or lose customer's trust
Subgroup Fairness	A fairness measure that ensures equitable treatment across all identifiable subgroups within a protected attribute	-	Intersectional fair ranking via subgroup divergence (2024)	Ensuring that approval rates for women are comparable to those for men, after accounting for financial factors
Intersectional Fairness	Fairness across combinations of protected categories (e.g., race and gender) to address multiple and intersecting forms of discrimination	-	Intersectional fair ranking via subgroup divergence (2024)	Ensuring equitable treatment for applicants who belong to multiple marginalized groups, such as Black women

Terms	Definition	Alternate Terms/Alias	Source	Example
Causal Fairness	Fairness criteria based on causal reasoning, ensuring decisions are not influenced by prohibited causal paths from protected attributes	-	On the Applicability of Machine Learning Fairness Notions (2021)	Factors like education or employment history influence the decision only through their legitimate impact on financial stability
Temporal Fairness	Fairness measured over time, ensuring consistent fairness as data and societal norms involving protected attributes evolve	-	Delayed Impact of Fair Machine Learning (2018)	Applicants with similar profiles receive similar decisions regardless of when they apply.
Robust Fairness	Ensuring that fairness properties hold under various conditions, including changes in the underlying population of protected attributes	-	Fairness Without Demographics in Repeated Loss Minimization (2018)	A loan approval system that consistently provides fair outcomes across various demographic groups even under different economic conditions or data perturbations

Table 4: Glossary of Fairness Concepts

Appendix B: Thesis Documentation

All thesis-related documents and artifacts can be found in the Git repository at *https://gitlab.uni-koblenz.de/rahulthiru/thesis*

Bibliography

[1] Avinash Agarwal, Harsh Agarwal, and Nihaarika Agarwal. „Fairness Score and Process Standardization: From Theory to Application“. en. In: *AI and Ethics* 3 (2023). DOI: 10.1007/s43681-022-00147-7. URL: https://doi.org/10.1007/s43681-022-00147-7.

[2] Muhammad Ali et al. „Discrimination through Optimization: How Facebook's Ad Delivery Can Lead to Biased Outcomes“. In: *Proc. ACM Hum.-Comput. Interact.* 3.CSCW (Nov. 2019). DOI: 10.1145/3359301. URL: https://doi.org/10.1145/3359301.

[3] Nagadivya Balasubramaniam et al. „Transparency and Explainability of AI Systems: From Ethical Guidelines to Requirements“. In: *Information and Software Technology* 159 (July 2023), p. 107197. DOI: 10.1016/j.infsof.2023.107197. URL: https://doi.org/10.1016/j.infsof.2023.107197.

[4] Niels Bantilan. „THEMIS-ML: A Fairness-Aware Machine Learning Interface for End-To-End Discrimination Discovery and Mitigation“. In: *Journal of Technology in Human Services* 36.1 (Jan. 2018), pp. 15–30. DOI: 10.1080/15228835.2017.1416512. URL: https://doi.org/10.1080/15228835.2017.1416512.

[5] Luciano Baresi, Chiara Criscuolo, and Carlo Ghezzi. „Understanding Fairness Requirements for ML-based Software“. en. In: *2023 IEEE 31st International Requirements Engineering Conference (RE)*. Sept. 2023. DOI: 10.1109/re57278.2023.00046. URL: https://doi.org/10.1109/re57278.2023.00046.

[6] Solon Barocas, Moritz Hardt, and Arvind Narayanan. „Fairness and Machine Learning Limitations and Opportunities“. In: 2018. URL: https://api.semanticscholar.org/CorpusID:113402716.

[7] R. K. E. Bellamy et al. „AI Fairness 360: An Extensible Toolkit for Detecting and Mitigating Algorithmic Bias“. In: *IBM Journal of Research and Development* 63.4/5 (2019), 4:1–4:15. DOI: 10.1147/JRD.2019.2942287.

[8] Richard Berk et al. „Fairness in Criminal Justice Risk Assessments: The State of the Art“. In: *Sociological Methods & Research* 50.1 (July 2018), pp. 3–44. DOI: 10.1177/0049124118782533. URL: https://doi.org/10.1177/0049124118782533.

[9] Rdp Binns. „Fairness in Machine Learning: Lessons from Political Philosophy“. In: *Journal of Machine Learning Research* (Jan. 2018). URL: https://ora.ox.ac.uk/objects/uuid:2ff2785b-b0d4-447a-8326-a1fcc4c80840.

[10] Sarah Bird et al. „FairLearn: A Toolkit for Assessing and Improving Fairness in AI“. en. In: *Microsoft* (May 2020). URL: https://www.microsoft.com/en-us/research/publication/fairlearn-a-toolkit-for-assessing-and-improving-fairness-in-ai/.

[11] Tolga Bolukbasi et al. „Man is to Computer Programmer as Woman is to Homemaker? Debiasing Word Embeddings“. In: *Proceedings of the 30th International Conference on Neural Information Processing Systems*. NIPS'16. Barcelona, Spain: Curran Associates Inc., 2016, pp. 4356–4364. ISBN: 9781510838819.

[12] Yuriy Brun and Alexandra Meliou. „Software fairness“. en. In: *ESEC/FSE 2018: Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. Oct. 2018. DOI: 10.1145/3236024.3264838. URL: https://doi.org/10.1145/3236024.3264838.

[13] Maarten Buyl and Tijl De Bie. „Inherent Limitations of AI Fairness“. In: *Communications of the ACM* 67.2 (Jan. 2024), pp. 48–55. DOI: 10.1145/3624700. URL: https://doi.org/10.1145/3624700.

[14] Simon Caton and Christian Haas. „Fairness in Machine Learning: A Survey“. In: *Association for Computing Machinery* 56.7 (Apr. 2024). ISSN: 0360-0300. DOI: 10.1145/3616865. URL: https://doi.org/10.1145/3616865.

[15] Zhisheng Chen. „Ethics and Discrimination in Artificial Intelligence-Enabled Recruitment Practices“. In: *Humanities and Social Sciences Communications* 10.1 (Sept. 2023). DOI: 10.1057/s41599-023-02079-x. URL: https://doi.org/10.1057/s41599-023-02079-x.

[16] Sam Corbett-Davies et al. „The Measure and Mismeasure of Fairness“. In: *Journal of Machine Learning Research* 24.1 (Mar. 2024). ISSN: 1532-4435.

[17] Hugo Cossette-Lefebvre and Jocelyn Maclure. „AI's fairness problem: understanding wrongful discrimination in the context of automated decision-making“. In: *AI and Ethics* 3.4 (2023), pp. 1255–1269.

[18] Paul Covington, Jay Adams, and Emre Sargin. „Deep Neural Networks for YouTube Recommendations“. In: *ACM Conference on Recommender Systems (RecSys '16)*. ACM, Sept. 2016. DOI: 10.1145/2959100.2959190. URL: https://doi.org/10.1145/2959100.2959190.

[19] Kate Crawford and Ryan Calo. „There is a Blind Spot in AI Research“. In: *Nature* 538.7625 (Oct. 2016), pp. 311–313. DOI: 10.1038/538311a. URL: https://doi.org/10.1038/538311a.

[20] Julia Dressel and Hany Farid. „The accuracy, fairness, and limits of predicting recidivism“. In: *Science Advances* 4.1 (Jan. 2018). DOI: 10.1126/sciadv.aao5580. URL: https://doi.org/10.1126/sciadv.aao5580.

[21] Jannik Dunkelau and Michael Leuschel. „Fairness-aware machine learning: An extensive overview“. In: *Universität Düsseldor: Düsseldorf, Germany* (2019).

[22] Bradley Eck et al. „A Monitoring Framework for Deployed Machine Learning Models with Supply Chain Examples“. In: *2022 IEEE International Conference on Big Data (Big Data)*. Dec. 2022. DOI: 10.1109/bigdata55660.2022.10020394. URL: https://doi.org/10.1109/bigdata55660.2022.10020394.

[23] Benjamin G. Edelman and Michael Luca. „Digital Discrimination: The Case of Airbnb.com“. In: *SSRN Electronic Journal* (Jan. 2014). DOI: 10.2139/ssrn.2377353. URL: https://doi.org/10.2139/ssrn.2377353.

[24] Jodie L. Ferguson, Pam Scholder Ellen, and William O. Bearden. „Procedural and Distributive Fairness: Determinants of Overall Price Fairness“. In: *Journal of Business Ethics* 121.2 (Apr. 2013), pp. 217–231. DOI: 10.1007/s10551-013-1694-2. URL: https://doi.org/10.1007/s10551-013-1694-2.

[25] Sorelle A. Friedler, Carlos Scheidegger, and Suresh Venkatasubramanian. „The (Im)possibility of Fairness“. In: *Communications of the ACM* 64.4 (Mar. 2021), pp. 136–143. DOI: 10.1145/3433949. URL: https://doi.org/10.1145/3433949.

[26] Yanbo Ge et al. „Racial Discrimination in Transportation Network Companies“. In: *Journal of Public Economics* 190 (Oct. 2020), p. 104205. DOI: 10.1016/j.jpubeco.2020.104205. URL: https://doi.org/10.1016/j.jpubeco.2020.104205.

[27] Sahin Cem Geyik, Stuart Ambler, and Krishnaram Kenthapadi. „Fairness-Aware Ranking in Search & Recommendation Systems with Application to LinkedIn Talent Search“. In: *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. KDD '19. Association for Computing Machinery, 2019, pp. 2221–2231. ISBN: 9781450362016. DOI: 10.1145/3292500.3330691. URL: https://doi.org/10.1145/3292500.3330691.

[28] Bachan Ghimire, Ze Shi Li, and Daniela Damian. „Understanding User Feedback in Software Ecosystems: A Study on Challenges and Mitigation Strategies“. In: *Software Business*. Springer Nature Switzerland, 2024, pp. 132–147. ISBN: 978-3-031-53227-6. DOI: 10.1007/978-3-031-53227-6_10.

[29] SOPHIST GmbH. *MASTER: Schablonen für alle Fälle*. en. 2016. URL: https://www.sophist.de/fileadmin/user_upload/Bilder_zu_Seiten/Publikationen/Wissen_for_free/RE-Broschuere_Englisch_-_Online.pdf.

[30] SOPHIST GmbH. *The SOPHISTs: A Short RE Primer*. en. 2016. URL: https://www.sophist.de/fileadmin/user_upload/Bilder_zu_Seiten/Publikationen/Wissen_for_free/RE-Broschuere_Englisch_-_Online.pdf.

[31] Katharina Großer et al. „Benchmarking Requirement Template Systems: Comparing Appropriateness, Usability, and Expressiveness“. In: *Requirements Engineering* (Aug. 2024). DOI: 10.1007/s00766-024-00427-0. URL: https://doi.org/10.1007/s00766-024-00427-0.

[32] Wei Guo and Aylin Caliskan. „Detecting Emergent Intersectional Biases: Contextualized Word Embeddings Contain a Distribution of Human-like Biases“. In: *Proceedings of the 2021 AAAI/ACM Conference on AI, Ethics, and Society*. AIES '21. New York, NY, USA: Association for Computing Machinery, 2021, pp. 122–133. ISBN: 9781450384735. DOI: 10.1145/3461702.3462536. URL: https://doi.org/10.1145/3461702.3462536.

[33] Varun Gupta et al. *Sustainability in Software Engineering and Business Information Management: Proceedings of the Conference SSEBIM 2022*. Springer Nature, May 2023.

[34] Xiaotian Han et al. „FFB: A Fair Fairness Benchmark for In-Processing Group Fairness Methods“. In: *The Twelfth International Conference on Learning Representations*. 2024. URL: https://openreview.net/forum?id=TzAJbTClAz.

[35] Jack Hardinges, Elena Simperl, and Nigel Shadbolt. „We Must Fix the Lack of Transparency around the Data Used to Train Foundation Models“. In: *Harvard Data Science Review* Special Issue 5 (Dec. 2023). DOI: 10.1162/99608f92.a50ec6e6. URL: https://doi.org/10.1162/99608f92.a50ec6e6.

[36] Moritz Hardt, Eric Price, and Nathan Srebro. „Equality of Opportunity in Supervised Learning“. en. In: *NIPS'16: Proceedings of the 30th International Conference on Neural Information Processing Systems*. 2016.

[37] Florian Heidecker, Maarten Bieshaar, and Bernhard Sick. „Corner Cases in Machine Learning Processes“. en. In: *AI Perspectives & Advances* 6.1 (2024). DOI: 10.1186/s42467-023-00015-y. URL: https://doi.org/10.1186/s42467-023-00015-y.

[38] Md. Hamid Hosen et al. „Enhancing Pneumonia Detection: CNN Interpretability with LIME and SHAP“. In: *6th International Conference on Electrical Engineering and Information & Communication Technology (ICEEICT)*. May 2024. DOI: 10.1109/iceeict62016.2024.10534430. URL: https://doi.org/10.1109/iceeict62016.2024.10534430.

[39] Elizabeth E. Joh. „Reckless automation in policing“. In: *SSRN Electronic Journal* (Jan. 2022). DOI: 10.2139/ssrn.4009911. URL: https://doi.org/10.2139/ssrn.4009911.

[40] Anna Katrine Jørgensen and Anders Søgaard. „Rawlsian AI Fairness Loopholes“. In: *AI and Ethics* 3.4 (Oct. 2022), pp. 1185–1192. DOI: 10.1007/s43681-022-00226-9. URL: https://doi.org/10.1007/s43681-022-00226-9.

[41] Mackenzie Jorgensen et al. „Investigating the Legality of Bias Mitigation Methods in the United Kingdom“. In: *IEEE Technology and Society Magazine* 42.4 (Dec. 2023), pp. 87–94. DOI: 10.1109/mts.2023.3341465. URL: https://doi.org/10.1109/mts.2023.3341465.

[42] Tonni Das Jui and Pablo Rivas. „Fairness Issues, Current Approaches, and Challenges in Machine Learning Models“. In: *International Journal of Machine Learning and Cybernetics* (Jan. 2024). DOI: 10.1007/s13042-023-02083-2. URL: https://doi.org/10.1007/s13042-023-02083-2.

[43] Faisal Kamiran and Toon Calders. „Data Preprocessing Techniques for Classification without Discrimination“. In: *Knowledge and Information Systems* 33.1 (Dec. 2011), pp. 1–33. DOI: 10.1007/s10115-011-0463-8. URL: https://doi.org/10.1007/s10115-011-0463-8.

[44] Toshihiro Kamishima et al. „Considerations on Fairness-Aware Data Mining“. In: *Proceedings of the 2012 IEEE 12th International Conference on Data Mining Workshops*. ICDMW '12. USA: IEEE Computer Society, 2012, pp. 378–385. ISBN: 9780769549255. DOI: 10.1109/ICDMW.2012.101. URL: https://doi.org/10.1109/ICDMW.2012.101.

[45] Michael Kearns et al. „Preventing Fairness Gerrymandering: Auditing and Learning for Subgroup Fairness“. In: *Proceedings of the 35th International Conference on Machine Learning*. Ed. by Jennifer Dy and Andreas Krause. Vol. 80. Proceedings of Machine Learning Research. PMLR, July 2018, pp. 2564–2572. URL: https://proceedings.mlr.press/v80/kearns18a.html.

[46] Krishnaram Kenthapadi et al. „Model Monitoring in Practice: Lessons Learned and Open Challenges“. In: *Proceedings of the 28th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. Association for Computing Machinery, 2022, pp. 4800–4801. ISBN: 9781450393850. DOI: 10.1145/3534678.3542617. URL: https://doi.org/10.1145/3534678.3542617.

[47] Niki Kilbertus et al. „Avoiding Discrimination through Causal Reasoning“. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Curran Associates Inc., 2017, pp. 656–666. ISBN: 9781510860964.

[48] Barbara Kitchenham, Stuart Charters, et al. „Guidelines for performing systematic literature reviews in software engineering version 2.3“. In: *Engineering* 45.4ve (2007), p. 1051.

[49] Jon M. Kleinberg, Sendhil Mullainathan, and Manish Raghavan. „Inherent Trade-Offs in the Fair Determination of Risk Scores“. In: *Conference on Innovations in Theoretical Computer Science*. Jan. 2017, p. 23. DOI: 10.4230/LIPIcs.ITCS.2017.43. URL: https://doi.org/10.4230/LIPIcs.ITCS.2017.43.

[50] Matt Kusner et al. „Counterfactual Fairness“. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Long Beach, California, USA: Curran Associates Inc., 2017, pp. 4069–4079. ISBN: 9781510860964.

[51] Clarisse Laupman, Laurianne-Marie Schippers, and Marilia Papaléo Gagliardi. „Biased Algorithms and the Discrimination upon Immigration Policy“. In: *Information Technology and Law Series*. Springer, Jan. 2022, pp. 187–204. DOI: 10.1007/978-94-6265-523-2_10. URL: https://doi.org/10.1007/978-94-6265-523-2_10.

[52] Scott M. Lundberg and Su-In Lee. „A Unified Approach to Interpreting Model Predictions“. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Long Beach, California, USA: Curran Associates Inc., 2017, pp. 4768–4777. ISBN: 9781510860964.

[53] Björn Lundgren and Nancy Abigail Nuñez Hernández. *Philosophy of Computing: Themes from IACAP 2019*. Springer Nature, May 2022.

[54] Olga Mack. „Promoting AI Fairness: The Application of Disparate Impact Theory“. In: *MIT Computational Law Report* (Aug. 2023). https://law.mit.edu/pub/promoting-ai-fairness-disparate-impact-theory.

[55] Trisha Mahoney, Kush R. Varshney, and Michael Hind. *AI Fairness: How to Measure and Reduce Unwanted Bias in Machine Learning*. IBM Journal of Research and Development, Jan. 2020.

[56] Nicolas Mayer et al. „An integrated conceptual model for information system security risk management supported by enterprise architecture management“. In: *Software Systems Modeling* 18.3 (Feb. 2018), pp. 2285–2312. DOI: 10.1007/s10270-018-0661-x. URL: https://doi.org/10.1007/s10270-018-0661-x.

[57] Ninareh Mehrabi et al. „A Survey on Bias and Fairness in Machine Learning“. en. In: *ACM Computing Surveys (CSUR)* 54.6115 (July 2022). DOI: 10.1145/3457607. URL: https://doi.org/10.1145/3457607.

[58] Chioma Ngozi Nwafor and Obumneme Zimuzor Nwafor. „Determinants of Non-Performing Loans: An Explainable Ensemble and Deep Neural Network Approach“. In: *Finance Research Letters* 56 (2023), p. 104084. ISSN: 1544-6123. DOI: 10.1016/j.frl.2023.104084. URL: https://www.sciencedirect.com/science/article/pii/S1544612323004567.

[59] Minsu Park, Chao Yu, and Michael Macy. „Fighting Bias with Bias: How Same-Race Endorsements Reduce Racial Discrimination on Airbnb“. In: *Science Advances* 9.6 (Feb. 2023). DOI: 10.1126/sciadv.add2315. URL: https://doi.org/10.1126/sciadv.add2315.

[60] Taeyoung Park et al. „Artificial Intelligence in Urologic Oncology: The Actual Clinical Practice Results of IBM Watson for Oncology in South Korea“. In: *Prostate International* 11.4 (Dec. 2023), pp. 218–221. DOI: 10.1016/j.prnil.2023.09.001. URL: https://doi.org/10.1016/j.prnil.2023.09.001.

[61] Juan Carlos Pereira-Kohatsu et al. „Detecting and Monitoring Hate Speech in Twitter“. In: *Sensors* 19.21 (Oct. 2019), p. 4654. DOI: 10.3390/s19214654. URL: https://doi.org/10.3390/s19214654.

[62] Geoff Pleiss et al. „On Fairness and Calibration“. In: *Proceedings of the 31st International Conference on Neural Information Processing Systems*. NIPS'17. Curran Associates Inc., 2017, pp. 5684–5693. ISBN: 9781510860964.

[63] Klaus Pohl. *Requirements Engineering Fundamentals, 2nd Edition: A Study Guide for the Certified Professional for Requirements Engineering Exam - Foundation Level - IREB compliant*. Rocky Nook, Inc., Apr. 2016.

[64] Ricardo Trainotti Rabonato and Lilian Berton. „A systematic review of fairness in machine learning“. In: *AI and Ethics* (2024), pp. 1–12.

[65] Inioluwa Deborah Raji and Joy Buolamwini. „Actionable Auditing“. en. In: *2019 AAAI/ACM Conference on AI, Ethics, and Society (AIES '19)*. Jan. 2019. DOI: 10.1145/3306618.3314244. URL: https://doi.org/10.1145/3306618.3314244.

[66] Q. Ramadan. *Software Fairness Engineering*. Presentation slides, University of Koblenz. 2023.

[67] Qusai Ramadan et al. „MBFair: a model-based verification methodology for detecting violations of individual fairness“. In: *Software Systems Modeling* (June 2024). DOI: 10.1007/s10270-024-01184-y. URL: https://doi.org/10.1007/s10270-024-01184-y.

[68] Marco Ribeiro, Sameer Singh, and Carlos Guestrin. „"Why Should I Trust You?": Explaining the Predictions of Any Classifier“. In: *ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '16)*. Jan. 2016. DOI: 10.18653/v1/n16-3020. URL: https://doi.org/10.18653/v1/n16-3020.

[69] Rashida Richardson, Jason Schultz, and Kate Crawford. „Dirty Data, Bad Predictions: How Civil Rights Violations Impact Police Data, Predictive Policing Systems, and Justice“. en. In: *New York University Law Review* (Feb. 2019).

[70] Colette Rolland and Naveen Prakash. „From conceptual modelling to requirements engineering“. In: *Annals of Software Engineering* 10.1 (2000), pp. 151–176.

[71] Chris Rupp. *Requirements Templates - the blueprint of your requirement*. en. 2014. URL: https://www.sophist.de/fileadmin/user_upload/Bilder_zu_Seiten/Publikationen/RE6/Webinhalte_Buchteil_3/Requirements_Templates_-_The_Blue_Print_of_your_Requirements_Rupp.pdf.

[72] Christian Sandvig et al. „Auditing Algorithms: Research Methods for Detecting Discrimination on Internet Platforms“. en. In: *A Preconference at the 64th Annual Meeting of the International Communication Association* (May 2014).

[73] Vivian Simon, Neta Rabin, and Hila Chalutz-Ben Gal. „Utilizing Data-Driven Methods to Identify Gender Bias in LinkedIn Profiles“. In: *Information Processing & Management* 60.5 (Sept. 2023), p. 103423. DOI: 10.1016/j.ipm.2023.103423. URL: https://doi.org/10.1016/j.ipm.2023.103423.

[74] Jessie J. Smith, Lex Beattie, and Henriette Cramer. „Scoping Fairness Objectives and Identifying Fairness Metrics for Recommender Systems: The Practitioners' Perspective“. In: *Proceedings of the ACM Web Conference 2023*. WWW '23. Austin, TX, USA: Association for Computing Machinery, 2023, pp. 3648–3659. ISBN: 9781450394161. DOI: 10.1145/3543507.3583204. URL: https://doi.org/10.1145/3543507.3583204.

[75] Harini Suresh and John V. Guttag. „A Framework for Understanding Unintended Consequences of Machine Learning“. en. In: *EAAMO '21: Proceedings of the 1st ACM Conference on Equity and Access in Algorithms, Mechanisms, and Optimization*. Jan. 2019. DOI: 10.1145/3465416.348330. URL: http://dblp.uni-trier.de/db/journals/corr/corr1901.html#abs-1901-10002.

[76] Austin Van Loon et al. „Negative Associations in Word Embeddings Predict Anti-black Bias across Regions–but Only via Name Frequency“. In: *Proceedings of the International AAAI Conference on Web and Social Media* 16 (May 2022), pp. 1419–1424. DOI: 10.1609/icwsm.v16i1.19399. URL: https://doi.org/10.1609/icwsm.v16i1.19399.

[77] Sahil Verma and Julia Rubin. „Fairness Definitions Explained“. In: *In Proceedings of the International Workshop on Software Fairness (FairWare '18)*. 2018. DOI: 10.1145/3194770.3194776. URL: https://doi.org/10.1145/3194770.3194776.

[78] Xiaomeng Wang, Yishi Zhang, and Ruilin Zhu. „A Brief Review on Algorithmic Fairness“. In: *Springer* 1.7 (2021). DOI: 10.1007/s44176-022-00006-z. URL: https://doi.org/10.1007/s44176-022-00006-z.

[79] Anne L. Washington. „How to Argue with an Algorithm: Lessons from the COMPAS ProPublica Debate“. en. In: *The Colorado Technology Law Journal* 17.13357874 (Feb. 2019).

[80] Hilde Weerts et al. „Fairlearn: Assessing and Improving Fairness of AI Systems“. In: *Journal of Machine Learning Research (JMLR)* 24.1 (Mar. 2024). ISSN: 1532-4435.

[81] James Wexler et al. „The What-If Tool: Interactive Probing of Machine Learning Models“. In: *IEEE Transactions on Visualization and Computer Graphics* (Jan. 2019), p. 1. DOI: 10.1109/tvcg.2019.2934619. URL: https://doi.org/10.1109/tvcg.2019.2934619.

[82] Kyra Yee, Uthaipon Tantipongpipat, and Shubhanshu Mishra. „Image Cropping on Twitter: Fairness Metrics, their Limitations, and the Importance of Representation, Design, and Agency“. In: *Proc. ACM Hum.-Comput. Interact.* 5.CSCW2 (Oct. 2021). DOI: 10.1145/3479594. URL: https://doi.org/10.1145/3479594.

[83] Brian Hu Zhang, Blake Lemoine, and Margaret Mitchell. „Mitigating Unwanted Biases with Adversarial Learning“. en. In: *Proceedings of the 2018 AAAI/ACM Conference on AI, Ethics, and Society*. Dec. 2018. DOI: 10.1145/3278721.3278779. URL: https://doi.org/10.1145/3278721.3278779.